

TECHNISCHE UNIVERSITÄT
CHEMNITZ

Title of the master thesis

Master Thesis

Submitted in Fulfilment of the
Requirements for the Academic Degree
M.Sc.

Dept. of Computer Science
Chair of Computer Engineering

Submitted by: Max Mustermann
Student ID: 111222
Date: 12.12.1212

Supervising tutor: Prof. Dr. W. Hardt
(further supervisors)

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam lobortis facilisis sem. Nullam nec mi et neque pharetra sollicitudin. Praesent imperdiet mi nec ante. Donec ullamcorper, felis non sodales commodo, lectus velit ultrices augue, a dignissim nibh lectus placerat pede. Vivamus nunc nunc, molestie ut, ultricies vel, semper in, velit. Ut porttitor. Praesent in sapien. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Duis fringilla tristique neque. Sed interdum libero ut metus. Pellentesque placerat. Nam rutrum augue a leo. Morbi sed elit sit amet ante lobortis sollicitudin. Praesent blandit blandit mauris. Praesent lectus tellus, aliquet aliquam, luctus a, egestas a, turpis. Mauris lacinia lorem sit amet ipsum. Nunc quis urna dictum turpis accumsan semper.

Keywords: Keyword1, Keyword2, Keyword3, ...max 5

Contents

Contents . . . 3

List of Figures . . . 6

List of Tables . . . 8

List of Abbreviations . . . 10

1 Introduction . . . 11
- 1.1 Background and Motivation . . . 11
- 1.2 Problem Statement . . . 12
- 1.3 Research Objectives . . . 13
  - 1.3.1 Primary Objective . . . 13
  - 1.3.2 Specific Objectives . . . 13
  - 1.3.3 Research Questions . . . 13
- 1.4 Thesis Structure . . . 14

2 State of the Art . . . 15
- 2.1 Activity Recognition in Manual Assembly . . . 15
- 2.2 Activity Monitoring in Industrial Practice . . . 16
- 2.3 Wrist-Worn Inertial Sensing . . . 17
- 2.4 Sound as a Cue for Hand and Tool Activity . . . 19
- 2.5 Fusion of Inertial and Acoustic Data . . . 20
- 2.6 Activity Recognition on Microcontrollers . . . 22
- 2.7 Evaluation Practice . . . 25
- 2.8 Summary and Research Gap . . . 26

3 Data Acquisition and Dataset Construction . . . 28
- 3.1 Sensor Platform and Computational Constraints . . . 28
- 3.2 Assembly Scenario and Activity Taxonomy . . . 29
- 3.3 Data-Collection Firmware . . . 30
  - 3.3.1 Dual-Stream UDP Architecture . . . 30
  - 3.3.2 Audio Double-Buffering and Packet Framing . . . 33
  - 3.3.3 Microsecond Timestamping and Sequence Numbering . . . 33
  - 3.3.4 Multi-Device Capture and Body Placements . . . 33
- 3.4 Recording Protocol . . . 34

3.5 Ground-Truth Annotation from Synchronised Video 35
3.5.1 Determination of the Audio-Video Offset 35
3.5.2 Interval Labels and Sample-Level Label Embedding 36
3.6 Signal Conditioning 36
3.6.1 Packet-Loss Detection and Audio-Dropout Reconstruction . . 38
3.6.2 IMU Decimation to 80 Hz 38
3.6.3 Audio Resampling to a Strict 16 kHz Timeline 39
3.7 Window Generation and Labelling 39
3.8 Dataset Statistics, Class Distribution, and Imbalance 41
3.9 Data-Quality Assessment and Known Artefacts 42
3.10 Chapter Summary 43

4 Model Design, Training and Deployment 45
4.1 Overview of the Experimental Design 45
4.2 Input Representations 47
4.2.1 Inertial Windows 47
4.2.2 Audio Log-Mel Spectrograms 47
4.3 Single-Modality Models 48
4.3.1 Inertial Model 48
4.3.2 Audio Model and Temporal Pooling Variants 49
4.4 Fusion Architectures 51
4.4.1 Early Fusion 51
4.4.2 Embedding-Level Fusion 52
4.4.3 Decision-Level Fusion 52
4.5 Classical Baseline 55
4.6 Deployment-Oriented Model Variants 55
4.7 Training Procedure 56
4.7.1 Software and Hardware Environment 56
4.7.2 Training Settings 57
4.8 Deployment on the Wearable Device 58
4.8.1 Target Platform and Software Stack 58
4.8.2 Model Conversion and Integer Quantization 59
4.8.3 On-Device Signal Processing 61
4.8.4 Firmware Structure and Inference Loop 61
4.8.5 Verification of the Deployment Pipeline 63
4.9 Evaluation Protocol 63
4.9.1 Grouped Train/Validation Split 63
4.9.2 Metrics 64
4.9.3 On-Device Measurements 65
4.10 Chapter Summary 65

5 Results and Evaluation 67

6 Discussion 68

7 Conclusion **69**

Bibliography **70**

List of Figures

2.1 Approaches for monitoring manual assembly. The dashed box marks the two sensor types used in this thesis. 16
2.2 Development of activity recognition with wrist-worn and body-worn motion and sound sensors. The dashed part of the time axis marks a gap of several years without closely related work. 19
2.3 Hardware classes for running activity recognition models, ordered by available working memory. Dots mark devices used in related work. The thick line marks the M5StickC Plus2 used in this thesis, from its 520 KB of internal SRAM to an additional 2 MB of external PSRAM. Memory ranges are typical values and not exact limits. Memory of the Arduino Nano 33 BLE Sense and the Jetson TX2 from the manufacturers' data sheets [CITATION]. 23

3.1 The M5StickC Plus2 wearable device. The ESP32-PICO-V3-02 system-on-chip, the six-axis IMU (MPU6886), and the microphone (SPM1423) are all integrated on the board. Image: M5Stack [21]. 29
3.2 Architecture of the multimodal data-acquisition system. Each wearable device (four in sessions 1–3, three in sessions 4–6) samples an IMU and a microphone independently, timestamps and frames the samples, and streams them as two separate UDP streams (inertial on one port, audio on another) to a single host, which separates the devices by packet source address and writes raw per-device files. . . . 32
3.3 Signal-conditioning and windowing pipeline, applied per device. The inertial and acoustic branches are brought to fixed rates before both are cut on the same window boundaries. 37
3.4 Window generation and majority-vote labelling. Two-second windows advance in one-second steps (W_1, W_2, W_3 shown stacked for clarity). A window takes the majority activity among its labelled samples; a window whose labelled coverage falls below 70 %, such as one spanning the transition region, is discarded. 40
3.5 Number of two-second windows per activity class in the dominant-wrist dataset ($n = 2565$). `screw_tightening` is the largest class with about 23 % of the windows, while `place_container` is the smallest with 74 windows. 42

4.1 Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models. 48

4.2 Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries. 50
4.3 The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models. 51
4.4 Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom). 54
4.5 Export of a trained model to the device firmware. All constants used on the device are generated automatically from the training pipeline. 60
4.6 Structure of the inference firmware. After the start-up phase, recording, preprocessing, inference and output are repeated continuously. . . 62
4.7 Grouped splitting. Long activity occurrences are cut into blocks, and whole blocks are assigned to either the training or the validation set. Training windows that overlap a validation window are discarded. A random split would distribute overlapping windows across both sets. . 64

List of Tables

2.1 Commercial systems that monitor manual work in production 17
2.2 Resource figures reported for activity recognition on embedded hardware. 24
2.3 Model types for activity recognition and their suitability for microcontrollers. 24
2.4 Effect of the evaluation protocol on reported results. 26
2.5 Comparison of closely related work. 27

3.1 Activity classes in the modelling dataset. Descriptions are taken from the 32-step assembly protocol. "Typ. dur." is the range of mean coded-segment durations across sessions 1–3; for `screw_tightening` it covers both the tightening and the merged loosening segments. "Win." is the number of two-second windows per class in the final dominant-wrist dataset (Section 3.8). 31
3.2 Body placements: data recorded during acquisition versus data used as model input in the final pipeline. Only the dominant-wrist inertial and acoustic streams reach the fusion models. 34
3.3 Data-acquisition sessions. "(+1)" denotes one further sequence marked invalid and set aside. Sessions 4–6 consist of longer bouts of individual activities instead of repetitions of the full sequence. 35
3.4 Composition of the dominant-wrist modelling dataset. 41
3.5 Data-quality indicators for the dataset. 43

4.1 Overview of the evaluated models. 46
4.2 Input representations of the two modalities. 48
4.3 Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations. 50
4.4 Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments. 53
4.5 Hand-crafted features of the classical baseline. 55
4.6 Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer. 57
4.7 Training settings shared by all neural network models. 57
4.8 Firmware variants for on-device inference. The tensor arena is the fixed memory block that holds the intermediate results of a network. 59

4.9 Values embedded in the firmware in addition to the model weights. . 60
4.10 Training, validation and discarded windows per class after the grouped split. 65

List of Abbreviations

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

KDE K Desktop Environment

SQL Structured Query Language

Bash Bourne-again shell

JDK Java Development Kit

VM Virtuelle Maschine

I²C Inter-Integrated Circuit

1 Introduction

1.1 Background and Motivation

Manual assembly remains a fundamental operation in industrial and manufacturing environments, and includes assembly tasks such as inserting parts, fastening screws, measuring, and component handling that are performed by human operators[CITATION]. In these environments, the automatic recognition of the operator's activity at a particular time, known as Human Activity Recognition (HAR), is of practical interest in several applications, such as process monitoring, ergonomic assessment, quality assurance, and training support [CITATION]. Sensor-based HAR, in which activities are classified based on signals captured by body-worn sensors rather than from video, has received particular attention because it avoids some of the privacy and line-of-sight limitations associated with camera-based systems [CITATION].

Inertial Measurement Units (IMUs) that combine accelerometers and gyroscopes are the most common sensing modalities for HAR, due to their small size, low cost, and ability to record motion patterns related to physical activity [CITATION]. However, a single inertial signal can be insufficient to distinguish between activities that share similar motion characteristics but differ in other respects. In a manual assembly context, this is a real limitation: many repetitive-motion activities with a hand tool, like fastening a screw, inserting a component, or striking with a tool, could have similar inertial signatures, while the sounds associated with these actions differ substantially. This motivates the use of audio as a complementary sensing modality alongside inertial sensing and the investigation of multimodal fusion of the two modalities.

In addition to the sensing modality, the way activity recognition is used affects its applicability. A system that requires constant access to an external computer or cloud service for making inferences ties activity recognition to external cloud compute infrastructure and network availability. A wearable device capable of performing inference locally, without network connectivity, is a more independent alternative. However, this requires the classification model to be implemented in the resource-constrained environment of low-cost, low-power hardware rather than on general-purpose computing hardware.

This thesis addresses these two considerations by investigating multimodal inertial and acoustic sensing to improve activity classification in a manual assembly context and by deploying the resulting classifier directly on a resource-constrained wearable device.

1.2 Problem Statement

One drawback of single-modality HAR systems based solely on inertial sensing is that they may not reliably distinguish between activities that require similar gross motor movements. There are a number of hand-proximal, repetitive-motion activities that are of this type within a given manual-assembly activity set, and their inertial signatures may not be well separated even though the sounds they produce may be quite different. This raises the question of whether and how to integrate audio with inertial data at the raw signal level, the level of learned representations, or at the level of model decisions to increase discriminability among these activities.

The second type of problem is not about accuracy but deployment. Offline evaluation of a classifier does not guarantee that the classifier will perform or even execute properly on an embedded target. When applying deep-learning models to a low-power wearable device, constraints on memory, computational resources, and inference latency must be considered. Models developed without these constraints may not be directly suitable for deployment. An on-device system also needs to replicate with high fidelity the signal processing performed when preparing the training data; any systematic difference between the processing done during training and during inference can lead to accuracy degradation, even if the underlying model is correct.

A further challenge arises from variations in sensor placement and orientation. Inertial measurements depend on the orientation of the sensor relative to the body and gravity. Consequently, a model trained under one sensor orientation may perform poorly when the device is worn in a different orientation during inference. Such a failure mode may not be apparent in an offline evaluation and may only become evident when the system is evaluated on-device under realistic operating conditions.

Overall, these issues highlight the need for more effective treatment of embedded multimodal HAR. Most published work on multimodal activity recognition focuses on how to evaluate the performance of fusion strategies without considering the question of whether fusion models can be deployed on the limited hardware they ultimately have to run on [CITATION]. However, research with embedded or on-device HAR often focuses on one type of sensor modality to avoid sensor fusion, thus avoiding the fusion question altogether [CITATION]. Comparatively little work has systematically evaluated different multimodal fusion strategies while also considering their adaptation and real-time execution on resource-constrained wearable devices. This thesis addresses this gap in the context of manual assembly activity recognition by evaluating multimodal fusion strategies together with their practical requirements for embedded deployment.

1.3 Research Objectives

1.3.1 Primary Objective

The main goal of this thesis is to design, implement and test a multimodal (IMU and audio) activity recognition system for manual assembly activities and deploy the most successful configuration(s) as real-time, on-device inference on a resource-constrained wearable device.

1.3.2 Specific Objectives

1. To design a data acquisition and preprocessing pipeline to synchronize inertial and audio streams from a wearable device and generate labeled fixed-length activity windows for supervised training.

2. To train and compare multiple sensor fusion strategies, from single-modality baselines, early (feature level) fusion, and late (embedding level and decision level) fusion, under a common evaluation protocol to assess which strategy is best suited for this activity set.

3. To make the best-performing model configurations suitable for embedded inference, including the on-device signal processing required to reproduce the preprocessing pipeline used during training.

4. To deploy and test several setups on the target wearable system, and to identify real-world limitations such as memory usage, latency, and sensitivity to the orientation of the sensors.

1.3.3 Research Questions

The following research questions are answered on the basis of the objectives above:

1. **Multimodal Sensor Contribution**: To what extent does adding audio to inertial sensing improve activity recognition performance for manual assembly tasks?

2. **Sensor Fusion Strategy**: Which fusion strategies provide the best activity recognition performance under a common preprocessing and evaluation protocol?

3. **Baselines**: How does multimodal fusion compare with single-modality baselines and classical machine-learning approaches based on hand-crafted features?

4. **Deployment viability**: Can a quantized multimodal fusion model be executed on a microcontroller-class wearable device within a latency suitable for real-time activity classification?

1.4 Thesis Structure

[To be added later]

2 State of the Art

This chapter gives an overview of the current state of research and practice for recognising assembly activities with sensors worn on the wrist. It starts with the application itself and with the way industry monitors manual assembly today. After that, it looks at the two sensor types used in this thesis, the inertial measurement unit and the microphone, and at the ways their data can be combined. The last sections deal with running such models on small embedded devices and with the question of how results in this field are evaluated. The chapter ends with a comparison of the most closely related work and the research gap that follows from it.

2.1 Activity Recognition in Manual Assembly

Even in highly automated factories, a large part of assembly is still done by hand [1]. If a system knows which step a worker is currently performing, it can be used in several ways. It can show new workers the right instruction at the right time, detect a missing or repeated step for quality assurance, or measure cycle times without a manual time study [34, 1]. The same information is also needed for ergonomic assessments and for human-robot collaboration, where the robot has to know what its human partner is doing [4]. An overview of these applications in production and logistics is given in [31].

Assembly activities are different from the activities that most activity recognition research deals with, such as walking, sitting or climbing stairs. They are short, often only a few seconds long, they are done mainly with the hands, and many of them involve a tool. Different activities can look very similar in terms of arm movement, for example turning a screwdriver and turning a wrench. In between the activities of interest there are longer phases of reaching, holding and waiting that do not belong to any class. In the literature these phases are called the null class [8].

There are two main ways to recognise such activities, which are shown in Figure 2.1. The first uses cameras that are mounted at the workstation and record RGB video, depth images or the skeleton of the worker. Deep learning models trained on such data can recognise the individual steps of an assembly task quite reliably, for example on the public HA4M dataset of a gear train assembly [23, 6]. However, cameras need a clear view of the hands, which is often blocked by the workpiece or by the worker's own body. They are also sensitive to viewpoint and lighting [1], and filming workers continuously raises privacy concerns. The second way uses sensors that are worn on the body. They move with the worker, need no

free line of sight and measure the movement directly where it happens. Their disadvantages are that the signal depends on how the sensor is worn, that the battery is small, and that the data has to be sent away or processed on a very small device. Large wearable datasets from industrial settings are still rare. One recent example is OpenPack, which contains more than 50 hours of packaging work recorded with wrist-worn and body-worn sensors, depth cameras and barcode scanners [39].

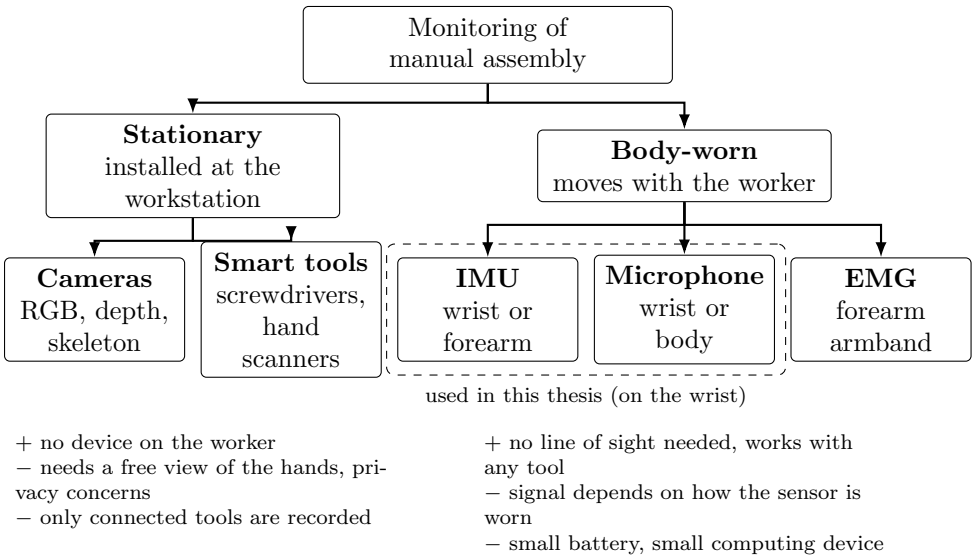


Figure 2.1: Approaches for monitoring manual assembly. The dashed box marks the two sensor types used in this thesis.

2.2 Activity Monitoring in Industrial Practice

The idea of using wearable sensors on the assembly line is not new. Already in 2008, a body-worn system for recognising worker activities was tested in two case studies with the car manufacturer Škoda [34]. One case study dealt with the training of new assembly workers and the other with quality assurance on the assembly line. However, such research prototypes have not led to general activity recognition products. The systems that are used in factories today record clearly defined events of special tools and devices instead. Table 2.1 lists some typical examples.

A well-known example are the wearable barcode scanners by ProGlove [30, 29]. The scanner is worn on the back of the hand, so the worker no longer has to pick up and put down a hand scanner for every part. Car manufacturers such as BMW and Volkswagen use these scanners in production and logistics, and the related software analyses scan and motion data to find long process times and ergonomic problems [30, 29]. In the assembly itself, screwing is monitored most closely, because a missing or loose screw is a typical quality problem. Fraunhofer IIS developed a sensor module that can be attached to an existing electric screwdriver [12]. From

Provider	Device	What is recorded
ProGlove [30, 29]	Barcode scanner worn on the back of the hand	Every scan of a part with a time stamp. Scan and motion data are used to analyse process times, bottlenecks and ergonomics. Used at BMW and Volkswagen.
Fraunhofer IIS with BMW [12]	Sensor module attached to an electric screwdriver	Rotational speed, acceleration and direction of the tool, used to check that every screw of a station was tightened. Tested at BMW Regensburg.
Bossard, SAP Digital Manufacturing [7, 32]	Connected screwdrivers and torque tools	The tool gets the next work step from the production system and reports torque, angle and result of every screw joint.
Ergonomics wearables, e.g. LifeBooster [27]	Inertial sensors worn on the upper body and arms	Posture and movement over a whole shift, used to estimate the risk of musculoskeletal disorders.

Table 2.1: Examples of commercial systems that monitor manual work in production. Information as given by the providers.

the rotational speed, acceleration and direction of the tool, the module detects each screwing operation. In a pilot project at the BMW plant in Regensburg, the worker was told after each car whether all screws had been tightened [12]. Connected screwdrivers, offered for example by Bossard and integrated into manufacturing software such as SAP Digital Manufacturing, go one step further. They receive the required torque from the production system and save the result of every screw joint [7, 32]. A different group of products uses inertial sensors on the body to assess ergonomic risks over a whole shift. A report of the U.S. Government Accountability Office points out, however, that the evidence for their benefit comes mostly from the vendors themselves [36].

All of these systems have one thing in common. They either record events that the tool or device itself produces, such as a scan or a finished screw joint, or they measure posture without knowing which task is being done. Work with tools that are not connected, such as a hammer, a manual screwdriver, a file or a tape measure, is not recorded at all, and every new tool would have to be equipped and integrated separately. A sensor on the worker's wrist could, in principle, cover all hand tools at once. This is the starting point for the research described in the following sections.

2.3 Wrist-Worn Inertial Sensing

Inertial measurement units (IMUs), which combine a three-axis accelerometer and a three-axis gyroscope, are the most common sensors for wearable activity recognition. They are small, cheap and energy-efficient, and they work without a line of sight [8,

18]. For assembly work the wrist is an obvious position. It is close to the hands, it moves with every tool operation, and most people are used to wearing a watch there. A smartwatch can tell apart 25 different hand activities, but only if it is worn on the arm that does the work [19]. In one of the first industrial studies, a single IMU was worn on the wrist of workers on an assembly line [17]. Its two-second windows of acceleration and angular velocity were classified with a k-nearest-neighbour classifier, and a state machine detected when a work task was finished. The detected start and end times differed from the true ones by about one second on average [17]. Other assembly studies use armbands on the forearm that also measure muscle activity with electromyography (EMG) [35, 1]. The forearm is close to the wrist, but an armband does not follow the rotation of the hand in the same way, so the two positions are not equivalent.

The basic processing steps for inertial data are well established. The continuous signal is cut into windows of fixed length, usually one to a few seconds long and overlapping by 50 %, and each window gets one activity label [8, 3]. Older systems calculated hand-crafted features for each window, such as mean, variance or frequency content, and classified them with methods like k-nearest neighbours or hidden Markov models [17, 37]. Random forests, which combine the votes of many decision trees, are still a common baseline for such features [9, 16]. Today, neural networks usually learn the features directly from the raw signal, and they often do better. In one comparison reported in [16], a simple CNN on smartphone accelerometer data reached 92.7 % accuracy, compared with 89.1 % for a random forest. One-dimensional convolutional neural networks (CNNs) move small learned filters along the time axis and detect typical motion patterns [35, 1]. If recurrent layers are added after the convolutions, as in the DeepConvLSTM architecture, the network can also learn in which order these patterns occur [40] [CITATION: Ordóñez and Roggen 2016]. Attention-based models go further and learn which parts of the signal are most important [14]. Recent surveys give an overview of these model types [18].

For assembly activities that differ clearly in their movement, the reported results are good. In one study, a CNN recognised six assembly actions, among them hammering, using a screwdriver and using a wrench, with an accuracy of 98 % when part of each worker's data was used for training, and 87 % when the test worker was not part of the training data at all [35] [VERIFY: figures against full text]. However, two problems remain. First, the signal depends on how the device is worn, so a model trained with one orientation of the device may fail with another [CITATION: sensor orientation and placement in wearable HAR]. Second, the movement of the wrist does not always show what the hand is actually doing. Turning a screwdriver, tightening a nut with a wrench and turning a knob all lead to a similar rotation of the forearm. Activities that mainly differ in the tool or the material are therefore hard to separate by movement alone. One way to partly solve this is to read the accelerometer of a smartwatch 4000 times per second instead of the usual 50 to 100 times. In this way the sensor also picks up the small vibrations that a tool or object transfers to the hand [19]. This is already a step towards sound.

2.4 Sound as a Cue for Hand and Tool Activity

Most tools make a typical sound. A saw makes a regular scraping noise, a hammer short loud impacts, and a drill or cordless screwdriver the humming of its motor. A microphone close to the hand records these sounds, together with the background noise of the workshop. The idea of combining a microphone and an accelerometer on the arm for this purpose goes back to the 2000s [20]. In the best-known study, Ward et al. recognised nine activities of an assembly task in a wood workshop, among them sawing, hammering and drilling [37]. The user wore microphones and accelerometers on the arms. The system compared the loudness of two microphones at different body positions to find the parts of the recording that contained an activity. The sound was then classified with linear discriminant analysis, the acceleration with hidden Markov models, and the results of both were combined. When the system was trained with data of the same user, it reached a recall of 78 % and a precision of 74 %. For a new user these values dropped to 66 % and 63 % [37]. The study showed that sound and wrist motion contain different and complementary information about tool use. It also showed how much harder the task becomes when the system has to work for people it has not seen before. Figure 2.2 shows how this line of research has developed since then.

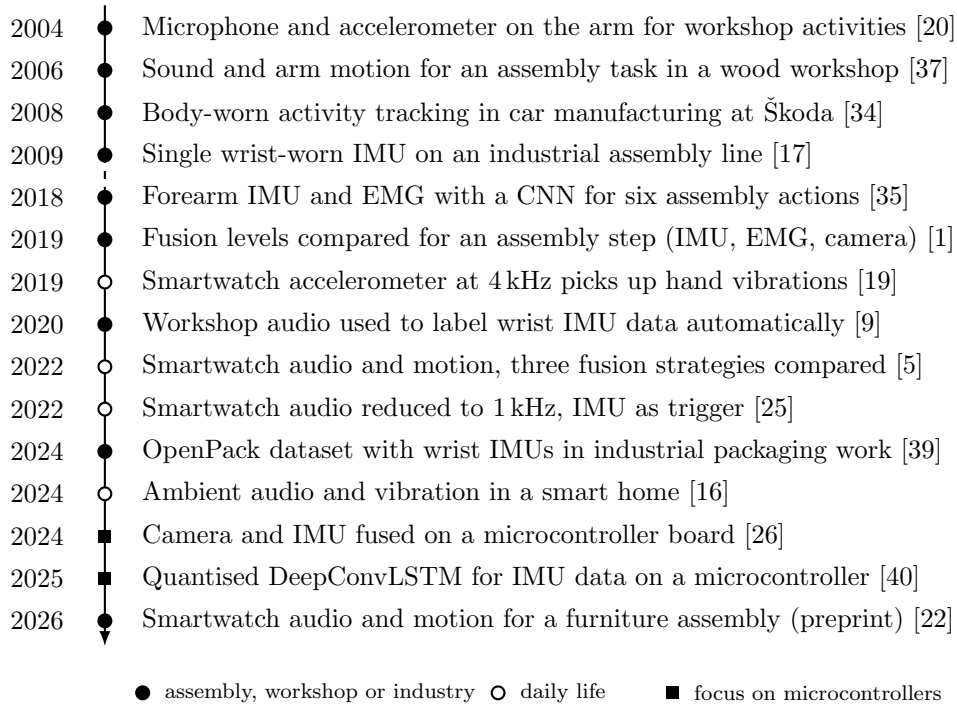


Figure 2.2: Development of activity recognition with wrist-worn and body-worn motion and sound sensors. The dashed part of the time axis marks a gap of several years without closely related work.

Since smartwatches contain both an IMU and a microphone, this combination

has received new attention, although mostly for activities of daily life. In one study, sound and motion were recorded with ordinary smartwatches from 15 participants in their own homes, and 23 activities such as cooking, writing and cleaning were recognised. Several models and three fusion strategies were compared, and the combination of both sensors worked better than each sensor alone. A second study under real everyday conditions showed, however, that recognition becomes much more difficult outside a controlled setting [5] [VERIFY: best fusion strategy in full text]. The SAMoSA system uses the IMU as a trigger and only switches on the microphone when movement is detected. In addition, the audio is recorded with a sampling rate of only 1 kHz, which saves energy and makes speech impossible to understand. The system recognised 26 activities in four indoor environments with an accuracy of 92.2 %, and reducing the audio sampling rate from 16 kHz to 1 kHz hardly changed the result [25]. A recent preprint uses audio and motion from a smartwatch to follow the steps of a furniture assembly, including screwing and drilling, and runs the system on edge devices without a cloud connection [22] [VERIFY: sensor setup, preprint].

Sound alone can also be a strong signal. In a smart home study with 25 activities and 14 participants, microphones placed in the rooms reached up to about 85 % accuracy, vibration sensors up to about 80 %, and the best results were achieved with audio or with audio and vibration together [16]. As in many audio classification systems, the sound was first converted into a mel spectrogram, an image-like representation of how the energy in different frequency bands changes over time, and then classified with a residual network, a deep CNN [16]. On construction sites, combining sound and motion data of construction machines improved the accuracy by up to 25 % compared with a single sensor [33]. Sound can also help indirectly. In the LASO system, participants wore a smartwatch for motion data while a smartphone in their pocket recorded audio during workshop activities such as sawing and hammering. A pretrained audio classifier then labelled the motion data automatically, and the audio was deleted afterwards [9].

The last example shows the main concern about microphones at the workplace. Audio can contain speech, and most workers will not accept being recorded all day. In Germany, the introduction of technical devices that can be used to monitor employees also requires the consent of the works council [CITATION: BetrVG § 87 (1) Nr. 6]. The approaches above deal with this problem in different ways. Audio is processed on the device and then deleted [9], the sampling rate is reduced so far that speech cannot be understood [25], or only features or recognised classes leave the device. Processing the data on the device itself is therefore not only a technical advantage but also important for the acceptance of such a system.

2.5 Fusion of Inertial and Acoustic Data

IMU and audio data are quite different. A wrist IMU delivers between some tens and a few hundred samples per second, a microphone several thousand. The IMU

signal is usually used as a time series with a few channels, while audio is mostly converted into a spectrogram, an image-like representation that shows how much energy each frequency band contains over time. Before the two signals can be combined, they also have to be aligned in time, for example by shifting them until their cross-correlation is highest [16]. The most important design decision is at which point in the processing chain the two signals are combined. Three levels are usually distinguished [1].

- **Early fusion** (data-level fusion) combines the raw or preprocessed signals into one input for a single model. This requires that both signals are brought into a common format.
- **Feature-level fusion** (embedding-level fusion) processes each signal with its own encoder network and combines the resulting feature vectors, called embeddings, before a common classifier.
- **Late fusion** (decision-level fusion) trains a separate classifier for each signal and combines their predicted probabilities, for example by taking the average or a weighted sum.

In addition, some models use attention mechanisms that learn how much each signal should contribute. These hybrid approaches do not fit clearly into one of the three levels [14, 26].

Only a few studies compare these levels on the same data with the same sensors. One of them deals with one step of a 3D printer assembly, recorded with IMU and EMG armbands on both forearms and a Kinect camera [1]. Five workers repeated the step ten times each. A separate CNN was trained for each sensor, and data-level, feature-level and score-level fusion were compared. A weighted score fusion, which gives each model more influence on the actions it recognises best, worked better than a simple average. For fine finger movements such as tightening a screw, the wearable sensors were more useful than the camera [1]. In the smart home study with room microphones and vibration sensors described in Section 2.4, the data of several sensor nodes was either merged into one input or processed separately with the predictions combined afterwards. The separate processing was clearly better, with up to 85 % instead of 75 % for audio and 80 % instead of 50 % for vibration [16]. For smartwatch audio and motion, three fusion strategies were compared in [5], and the early work on assembly activities already tested several ways of combining the decisions of the sound and motion classifiers [37] [VERIFY: fusion methods in the full text].

Several studies report that a simple combination does not always make good use of both signals. In late fusion, one signal can dominate the result while the other one hardly contributes, an effect called modality collapse [6]. Possible solutions are to pretrain the encoder for each signal separately and fuse the features in several stages [10], or to use contrastive pretraining, where the model learns that recordings of both signals from the same moment belong together. The latter also helps when

only little labelled data is available [28]. Attention can also be used to give the IMU more weight for activities with body movement and a second sensor more weight for activities without movement [26].

The choice of fusion level also affects the computational cost. Early fusion needs only one network, but both signals have to fit into one common input format. Feature-level and late fusion keep a separate network for each signal, so memory and computing time increase. One way to reduce this cost is to convert the IMU signal into an image, so that the same network can process both signals [14]. No study was found that compares the fusion levels for audio and wrist motion and also reports how much memory and time each variant needs on the same embedded hardware.

2.6 Activity Recognition on Microcontrollers

The term edge device is used for very different hardware. Single-board computers such as the Raspberry Pi or the NVIDIA Jetson run a full Linux system and have several gigabytes of memory, and smartwatches and smartphones are similar in this respect. Microcontrollers are much smaller. They run without an operating system or with a small real-time system, have a few hundred kilobytes to a few megabytes of memory and run at some hundred megahertz. Compared with smartphones, they have about 100 to 1000 times less computing power, memory and energy [11]. Figure 2.3 shows these hardware classes by their available memory, together with the devices used in related work. Running neural networks on such devices is called TinyML. The most widely used framework is TensorFlow Lite Micro. It executes a converted model using a fixed block of memory, the so-called tensor arena, which is reserved once at the start, and it only includes the operations that the model actually needs [11]. To make the model smaller and to allow integer arithmetic, the weights and intermediate values are usually converted from 32-bit floating-point numbers to 8-bit integers. This step is called quantisation. A small set of typical input examples is used to find out the value range of each layer [40].

For models that use only IMU data, this approach is well documented. In one example, a DeepConvLSTM was trained on the WISDM dataset, which contains accelerometer data from a smartphone in the trouser pocket, and ran on an Arduino Nano 33 BLE Sense microcontroller. Quantisation reduced the model size from 513 KB to 137 KB, while the accuracy dropped only from 98.2 % to about 97 %. On the device, the model needed 29 KB of RAM and 190 KB of flash memory, and one prediction took 21 ms [40]. Multimodal models are rare on this kind of hardware. One example combines a chest-worn camera and an IMU with an attention mechanism on a Sony Spresense board with about 2 MB of RAM and needs about 21 ms per prediction. The model size, quantisation and memory usage are not reported, however [26]. Many multimodal systems are only tested on single-board computers such as the Raspberry Pi or the NVIDIA Jetson [9, 14, 28]. Table 2.2 summarises the resource figures that these studies report. It shows that the figures are often

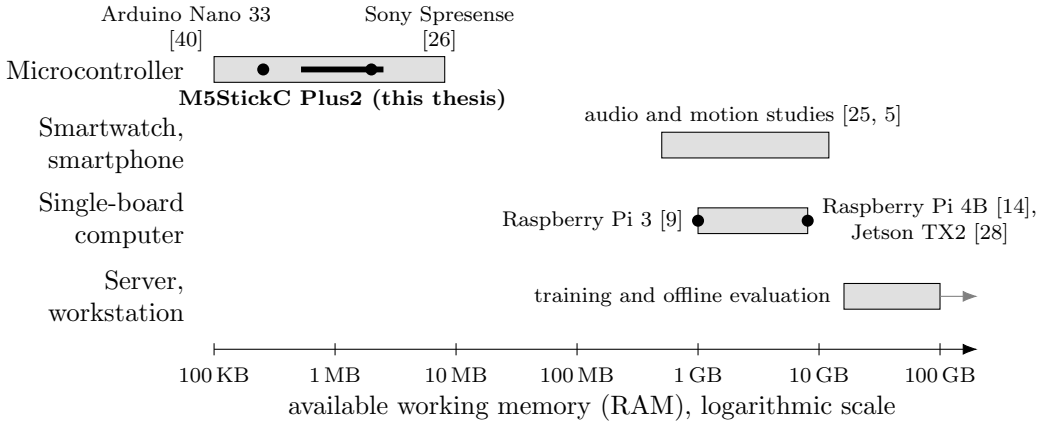


Figure 2.3: Hardware classes for running activity recognition models, ordered by available working memory. Dots mark devices used in related work. The thick line marks the M5StickC Plus2 used in this thesis, from its 520 KB of internal SRAM to an additional 2 MB of external PSRAM. Memory ranges are typical values and not exact limits. Memory of the Arduino Nano 33 BLE Sense and the Jetson TX2 from the manufacturers' data sheets [CITATION].

incomplete and that the models on single-board computers are several orders of magnitude larger than those on microcontrollers.

Which type of model is suitable for a microcontroller depends not only on its accuracy but also on its size and on the operations it needs. Table 2.3 gives an overview of the model types discussed in this chapter. Hand-crafted features with a random forest need very little memory, but the features have to be designed by hand for every sensor [9, 16]. CNNs are the most common compromise. They learn the features themselves, need few parameters because the same small filters are applied across the whole window, and consist of operations that TensorFlow Lite Micro supports well [11, 40]. Recurrent layers such as LSTM or GRU additionally capture the order of patterns in a window, and they do not have to be large. In a direct comparison on a microcontroller, a DeepConvLSTM with about 130 000 parameters was both more accurate (98.2 %) and smaller after quantisation (137 KB) than a one-dimensional CNN with about 378 000 parameters (96.9 %, 382 KB) [40]. However, recurrent layers have to process the time steps one after the other, and TensorFlow Lite Micro supports only a subset of the operations of the full framework [11], so recurrent layers often have to be rewritten into simpler operations before they can run on the device [CITATION]. Their benefit therefore has to be weighed against their cost in each case. Attention-based and transformer models, finally, are mainly used for multimodal recognition with video [14, 6]. They are much larger. MMTSA, for example, has about 32 million parameters and needs 5.8 s per input on a Raspberry Pi, which is far beyond the limits of a microcontroller [14].

Audio models on microcontrollers are common for keyword spotting, where a

Device	**Sensors**	**Model size**	**Memory used**	**Time per prediction**
Arduino Nano 33 BLE Sense (MCU) [40]	Accelerometer	137 KB, 8-bit integer	29 KB RAM, 190 KB flash	21 ms
Sony Spresense (MCU) [26]	Camera, IMU	not reported	not reported	about 21 ms
Raspberry Pi 3 B (SBC) [9]	IMU, audio	not reported	CPU and memory load shown, no model size	slower than on a laptop[a]
Raspberry Pi 4B (SBC) [14]	Video, accelerometer	32.4 million parameters, floating point	not reported	5.8 s for a 15 s input
NVIDIA Jetson TX2 (SBC) [28]	Depth camera, IMU, radar	not reported	not reported	below 0.4 s for a 2 s input

MCU = microcontroller, SBC = single-board computer. [a] Execution time of the whole labelling process, not of a single prediction.

Table 2.2: Resource figures reported for activity recognition on embedded hardware.

Model type	**Typical input**	**Suitability for a microcontroller**	**Examples**
Hand-crafted features with a random forest	statistics per window, any sensor	very small, but features must be designed by hand	[9, 16]
1D CNN	raw IMU windows	small, simple operations that are well supported	[35, 1, 40]
2D CNN on a (mel) spectrogram	audio	small to medium, but the spectrogram has to be computed on the device	[16]
CNN with recurrent layers (e.g. DeepConvLSTM)	raw IMU windows	can be small, but processes time steps one after the other and needs operations with limited support	[40]
Attention and transformer models	video combined with IMU	very large (e.g. 32 million parameters [14]), so far only on single-board computers or larger	[14, 6]

Table 2.3: Model types for activity recognition and their suitability for microcontrollers.

small CNN detects spoken commands in a spectrogram of about one second of audio. The standard datasets and example applications for this task use a sampling rate of 16 kHz [38] [CITATION: keyword spotting on microcontrollers]. If a model works on spectrograms, the feature extraction, which consists of cutting the signal into short frames, computing a Fourier transform and applying a mel filter bank, also has to run on the microcontroller. It has to give the same values as the implementation used for training, otherwise the model receives inputs it has never seen. For models that work directly on raw IMU windows, as in the example above, this step is much simpler [40]. When audio and motion are combined on a wearable, however, the microphone is usually the part that uses the most energy, which is why SAMoSA only switches it on when needed [25]. The smartwatch systems for audio and motion described in Section 2.4 run on smartwatches and phones or are evaluated offline. No study was found that runs and compares several fusion variants of audio and wrist motion on a microcontroller.

2.7 Evaluation Practice

How well an activity recognition system seems to work depends strongly on how the data is split into training and test data. Since neighbouring windows usually overlap by half, they share a large part of their samples. If windows are assigned to the training and test set randomly, the test set contains windows that are almost identical to windows in the training set, and the measured accuracy is higher than it would be for new recordings. It has been shown that this effect leads to a clear overestimation of performance for many different features and classifiers, and that long, repetitive activities are affected most [15]. To avoid it, the data should be split into continuous blocks instead of single windows.

A second issue is how well a system works for new people. Results are always higher when data of the same person is used for training and testing than when the test person is completely left out. Table 2.4 shows this effect for several studies. For the 3D printer assembly, for example, the best fusion model reached 84.6 % when half of the windows of each worker were used for training, but only 77.8 % when the test worker was left out, and the best single-sensor model reached only about 65 % in this case [1].

Many studies only give the ratio of the split, for example 80/20, without saying whether the split was made by person, by session or by window [40, 16, 23]. This makes it hard to compare their numbers. Datasets from assembly and workshop settings are also small, often with only about five participants [37, 1, 9], so the evaluation protocol has a large effect on the result. Some methods are designed especially for this situation and are tested with only one to five percent of the data labelled [28].

Finally, assembly datasets are usually imbalanced, because some activities take much longer than others. The accuracy then mainly shows how well the frequent classes are recognised. For this reason the macro F1 score is often reported in addi-

Task	**Metric**	**Same persons in training and test**	**Test persons left out**
Wood workshop assembly, 9 activities [37]	Recall / precision	78 % / 74 %	66 % / 63 %
Assembly actions with forearm IMU and EMG, 6 actions [35]	Accuracy	98 %	87 %[a]
3D printer assembly step, 7 actions, best fusion model [1]	Accuracy	84.6 %	77.8 %
Daily activities (MMAct), video and IMU [14]	F1 score	94.07 %[b]	87.41 %

[a] [VERIFY: figures against full text]. [b] Split by recording session, so the same persons appear in training and test.

Table 2.4: Effect of the evaluation protocol on reported results.

tion to accuracy. It is calculated from precision and recall for each class separately and then averaged, so that every class counts the same [8].

2.8 Summary and Research Gap

Table 2.5 compares the work that is most closely related to this thesis. For each system it shows whether the IMU is worn on the wrist, whether audio is used as an input for classification, whether the activities involve assembly or tool use, whether different fusion strategies are compared, and whether the model runs on a microcontroller.

Each of these aspects has been studied on its own. Assembly and tool activities have been recognised with wrist-worn IMUs since the late 2000s [37, 17], and these early studies already showed that sound recorded close to the hands adds information that movement alone does not provide. Studies with smartwatches have confirmed that audio and wrist motion complement each other, and different fusion strategies for these two signals have been compared. IMU models are regularly quantised and run on microcontrollers. In industry, on the other hand, assembly is monitored with connected tools and scanners, and there is no general wearable that recognises work with any hand tool.

What is missing is a combination of these aspects. The studies that compare fusion strategies for audio and motion deal with daily activities and run on smartwatches or offline. The assembly studies that compare fusion levels use EMG and cameras instead of sound. The multimodal systems that run on a microcontroller use only one fusion approach and report their resource needs only in part. It is therefore still open which fusion strategy for audio and wrist motion is best suited for assembly activities if the model has to run on the wearable device itself, and how much

System	Wrist-worn IMU	Audio as input	Assembly or tool use	Fusion levels compared	Runs on MCU
Ward et al. 2006 [37]	yes	yes	yes	partly[a]	no
Stiefmeier et al. 2008 [34]	partly[b]	no	yes	no	no
Koskimäki et al. 2009 [17]	yes	no	yes	–	no
Tao et al. 2018 [35]	partly[c]	no	yes	no	no
Al-Amin et al. 2019 [1]	partly[c]	no	yes	yes	no
Laput and Harrison 2019 [19]	yes	partly[d]	no	–	no
LASO 2020 [9]	yes	partly[e]	partly[f]	no	no
SAMoSA 2022 [25]	yes	yes	no	no	no
Bhattacharya et al. 2022 [5]	yes	yes	no	yes	no
OpenPack 2024 [39]	yes	no	partly[g]	no	no
Koch et al. 2024 [16]	no	yes	no	partly[h]	no
Nakabayashi and Saito 2024 [26]	no	no	no	no	yes
Zhou et al. 2025 [40]	no	no	no	–	yes
Mahfuz et al. 2026 [22]	yes	yes	yes	no	no[i]
This thesis	yes	yes	yes	yes	yes

"–" not applicable because only one sensor type is used. [a] Several ways of combining classifier decisions, no early fusion. [b] Several body-worn sensors, not one wrist device. [c] Forearm armband with IMU and EMG. [d] Accelerometer with a high sampling rate that picks up vibrations, no microphone. [e] Audio from a phone in the pocket, only used to label the IMU data. [f] Workshop activities such as sawing and hammering, no assembly sequence. [g] Packaging work in logistics. [h] Merged versus separately processed sensor nodes. [i] Runs on edge devices, hardware type not stated in the abstract.

Table 2.5: Comparison of closely related work.

memory and computing time the different strategies need on such hardware. This thesis addresses this question. Chapter 3 describes the dataset of assembly activities recorded with wrist-worn devices, and Chapter 4 describes the fusion models and their deployment on a microcontroller.

3 Data Acquisition and Dataset Construction

This chapter explains the process used to create the multimodal dataset used in the remainder of this thesis. It introduces the wearable sensing platform and its resource envelope, the assembly scenario, the activity classes recorded, the firmware that streamed inertial and acoustic data off the devices, the protocol used to record participants, the video-based annotation procedure, and the steps of signal conditioning and windowing that transform the raw streams into labelled two-second segments. The chapter ends with a quantitative characterisation of the resulting dataset and an assessment of its known limitations. The modelling choices made on this dataset are deferred to Chapter 4, and the implications of the dataset's limitations for the validity of the results are addressed in Chapter 6.

3.1 Sensor Platform and Computational Constraints

The sensing platform is the M5StickC Plus2, a self-contained wearable device built around the ESP32-PICO-V3-02 system-on-chip (SoC), a dual-core processor clocked up to 240 MHz [21]. Each device is equipped with a six-axis inertial measurement unit (IMU, MPU6886) providing three-axis acceleration and three-axis angular velocity, a digital MEMS microphone (SPM1423), a small display, a single-cell 200 mAh lithium-polymer battery, and a 2.4 GHz Wi-Fi radio [21].

Two properties of this platform shape the entire data pipeline. The first is that the device is worn on the body, which rules out a high-bandwidth tethered connection and makes wireless streaming the practical way to obtain data from several body locations simultaneously. The second is the limited on-chip memory: the manufacturer datasheet gives 8 MB of flash, 2 MB of in-package PSRAM, and 520 KB of on-chip SRAM. This budget is comfortable for buffering and forwarding sensor samples during acquisition, but it is the binding constraint on the eventual on-device inference stage treated in Section 4.8. The dataset was therefore designed so that the same device that records the training data can, in principle, run the trained model: the inertial and acoustic sampling rates, the window length, and the feature representations described later in this chapter were all chosen with that target in mind rather than for offline convenience.

During data collection both the microphone and the IMU were used. The microphone is single-channel, so each device produced one monophonic audio stream alongside its six-axis inertial stream.



Figure 3.1: The M5StickC Plus2 wearable device. The ESP32-PICO-V3-02 system-on-chip, the six-axis IMU (MPU6886), and the microphone (SPM1423) are all integrated on the board. Image: M5Stack [21].

3.2 Assembly Scenario and Activity Taxonomy

Recordings were carried out in the Future Work Lab at Fraunhofer IPA in Stuttgart, a laboratory configured for research on human–machine interaction in manufacturing [CITATION]. The environment provides a workbench with parts, hand tools, and fasteners, so that a manual assembly task can be performed under controlled but realistic conditions.

Data were recorded from two participants over six sessions, all in the same laboratory and with the same equipment. In sessions 1–3, one participant wore the sensors and performed the full assembly task while the other filmed with a handheld mobile-phone camera, following the acting participant on foot during the segments that involved walking. The participants exchanged roles between sessions. Each of these sessions consisted of a repeated assembly sequence, with three repetitions in two of the sessions and four repetitions plus one sequence marked invalid in the third. These sessions lasted approximately 6.7, 7.6 and 16.9 minutes.

Sessions 1–3 produced few examples of the short or infrequent activities, such as placing a container or walking, while screwdriving dominated. Sessions 4–6 were therefore recorded with a different focus. Instead of the complete assembly sequence, the participant performed individual activities from the same task as longer continuous bouts, for example several minutes of walking or carrying a container. These sessions lasted approximately 12.1, 16.5 and 8.3 minutes and bring the total recording time to about 68 minutes, as summarised in Section 3.4.

Each repetition followed the same 32-step protocol, transcribed in the annotation records. The participant begins at the bench, walks to a supply table, and carries parts back; stacks three parts so that they align; seats them with a hammer; measures the assembly with a scale; passes four screws through the aligned parts by hand; drives the screws with a powered screwdriver; and places the finished item

in a storage compartment beside the bench. The remainder of each repetition reverses the process, loosening and removing the screws and returning the parts to their boxes. Because the same operations recur several times per repetition, the screwdriving class dominates the dataset (Section 3.8).

Video of sessions 1–3 was annotated with a larger set of labels than the dataset finally uses. Across these sessions the coded label set contained fifteen to eighteen activity labels, including narration (`speech`), the disassembly actions (`screw_loosening`, `remove_screws`, `store_part`), a rare `pick_container` action, and in one session the postures `sit` and `stand`. The modelling taxonomy retains the nine classes listed in Table 3.1, which cover the assembly half of the task together with locomotion and rest.

`screw_loosening` was merged into `screw_tightening`. The two actions use the same powered screwdriver and differ only in the direction of rotation, which produces nearly identical inertial and acoustic signatures that a two-second window cannot reliably separate. The class `compartment_interaction`, which describes opening and closing the storage compartment beside the bench, was coded in sessions 1–3 but not included in the final taxonomy. The movement involved is short and unspecific, and the compartment produces no characteristic sound, so the class offers little that distinguishes it from other activities in either modality. In addition, no further examples of it were recorded in sessions 4–6, so its data remained limited. The remaining labels (`speech`, `remove_screws`, `store_part`, `pick_container`, and, in one session, `sit`/`stand`) were excluded rather than merged.

The class descriptions in Table 3.1 are operational summaries taken from the 32-step protocol. The typical instance duration is the mean length of a coded activity segment in sessions 1–3, since sessions 4–6 contain much longer bouts.

3.3 Data-Collection Firmware

The same custom firmware was installed on all of the devices, written in C++ and based on the Arduino framework. Its only task during acquisition is to sample the two sensors and forward every sample to a host computer with enough timing information for the streams to be reassembled and aligned afterwards. It performs no filtering, no feature extraction, and no on-device storage. This section explains the data acquisition architecture and the decisions behind it, rather than the source code.

3.3.1 Dual-Stream UDP Architecture

The inertial and acoustic data are transmitted as two separate streams over Wi-Fi using the User Datagram Protocol (UDP). One port receives inertial packets and another receives audio packets, so that a receiver does not need to inspect packet contents to separate the two modalities. UDP was preferred to a connection-oriented protocol since it introduces no retransmission or reordering delay, keeping the sender

Class	Operational description (from the recording protocol)	Typ. dur.	Win.
`idle`	Standing at the bench without performing a task.	4–12 s	307
`walk`	Walking between the workstation and the supply table.	≈5 s	386
`carry_container`	Carrying the parts container between the supply table and the workstation.	5–9 s	220
`place_container`	Setting the parts container down on the work surface.	3–5 s	74
`assemble_parts`	Stacking three parts on the workstation so that they align and fit together.	10–19 s	166
`hammering`	Striking the stacked parts with a hammer to seat them.	6–11 s	250
`measuring`	Measuring the length and width of the assembled part with a scale.	8–9 s	320
`insert_screws`	Passing four screws through the aligned parts by hand.	19–22 s	246
`screw_tightening`	Driving screws with a powered screwdriver, including loosening actions merged into this class (Section 3.2).	22–31 s	596
Total paired windows			**2565**

Table 3.1: Activity classes in the modelling dataset. Descriptions are taken from the 32-step assembly protocol. "Typ. dur." is the range of mean coded-segment durations across sessions 1–3; for `screw_tightening` it covers both the tightening and the merged loosening segments. "Win." is the number of two-second windows per class in the final dominant-wrist dataset (Section 3.8).

simple and the timing of transmitted packets close to the timing of the underlying samples; the trade-off is that lost packets are not retransmitted by the transport and must be detected and handled in post-processing [CITATION].

Each time a new IMU sample becomes available, an inertial packet is created that contains the six inertial values as text, a sequence number, and a device timestamp. Each audio packet has a sequence number, a sample count, a device timestamp, and a fixed-size block of pulse-code-modulated (PCM) samples. Since multiple devices send to the same host on the same two ports, the receiver logs the source address of each packet it receives and uses it to differentiate the streams by device. Figure 3.2 shows the end-to-end acquisition path.

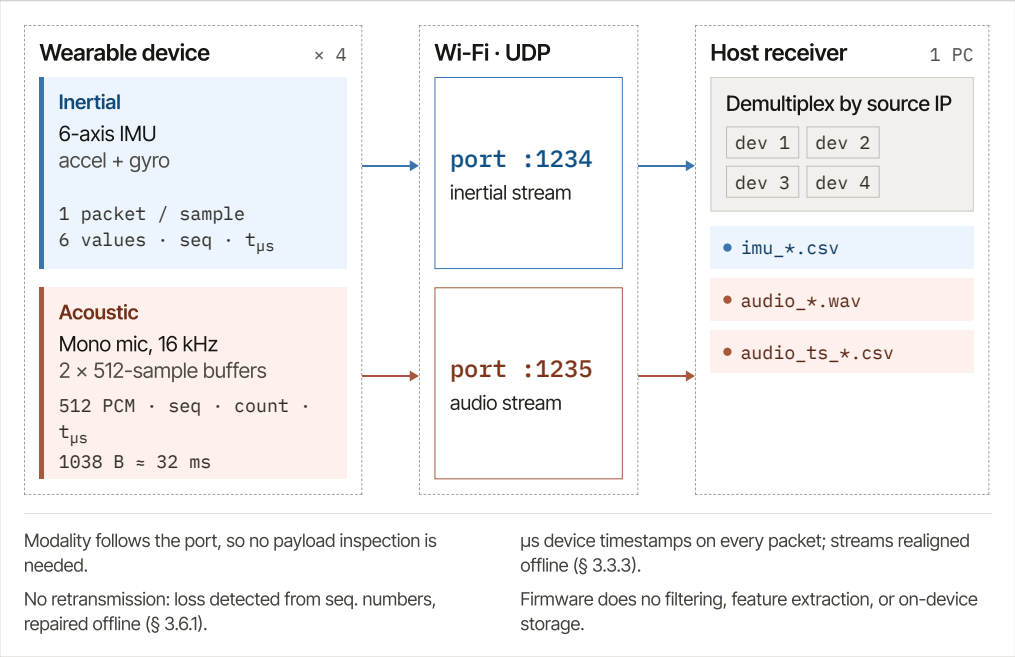


Figure 3.2: Architecture of the multimodal data-acquisition system. Each wearable device (four in sessions 1–3, three in sessions 4–6) samples an IMU and a microphone independently, timestamps and frames the samples, and streams them as two separate UDP streams (inertial on one port, audio on another) to a single host, which separates the devices by packet source address and writes raw per-device files.

3.3.2 Audio Double-Buffering and Packet Framing

Audio is captured continuously, so the firmware cannot stall the microphone driver while a packet is being transmitted. To avoid this, two 512-sample buffers are used alternately: while the driver fills one buffer, the previously filled buffer is transmitted, and once a buffer is full, the roles swap. This prevents new samples from overwriting data that has not yet been transmitted and keeps the capture cadence regular.

The buffer size also determines the size of each audio packet. A 512-sample block at a 16 kHz sampling rate corresponds to 32 ms of audio. Together with a header carrying the sequence number, sample count, and timestamp, this produces a packet of 1038 bytes, which is well below the 1500-byte Ethernet MTU (Maximum Transmission Unit), so every audio packet can be transmitted as a single, unfragmented UDP datagram. Keeping packets under the MTU matters because IP fragmentation of a UDP datagram means that losing any one fragment discards the whole datagram, which can increase the effective audio packet loss rate [CITATION].

3.3.3 Microsecond Timestamping and Sequence Numbering

Each packet, inertial and acoustic, carries a timestamp taken from the device's microsecond counter at the moment the packet is assembled, together with a monotonically increasing sequence number. These two fields serve different purposes. The sequence number makes packet loss detectable: a gap between consecutive received sequence numbers means a lost packet, independent of any timing assumption. The timestamp makes the true timing of a recording session recoverable: it allows the elapsed time between the first and last samples to be determined, which is later used for clock-drift correction in Section 3.6. It also provides the temporal reference needed to align the inertial and acoustic streams from the same device.

The microsecond resolution is more than sufficient for this purpose: it is far finer than the interval between individual inertial samples and than the interval between consecutive audio packets, so timestamp quantisation does not introduce a meaningful limitation to the alignment accuracy. The devices' timestamps cannot be compared directly with one another, however, since the individual devices do not share a common clock. The video reference described in Section 3.5 is therefore used to synchronise the various devices.

3.3.4 Multi-Device Capture and Body Placements

In sessions 1–3, four sensor devices were worn: one on the dominant wrist, one on the non-dominant wrist, one at the waist, and one on the leg. In sessions 4–6, the leg device was omitted, since its data had proved of little use for the bench-based activities, as explained below. All devices simultaneously recorded inertial and acoustic data and streamed them to the host system. Recording at several body locations was a deliberate design choice. It made it possible to capture potentially

useful information from different parts of the body and offered redundancy against a device failing during a recording session.

However, not all of the recorded data were used to train the models. Table 3.2 summarises the sensor data recorded and the data finally used. All models use the inertial and acoustic data of the dominant wrist. The only exception is the standalone audio model, which additionally uses the audio of the non-dominant wrist and the waist from sessions 4–6 to increase the amount of acoustic training data, as described in Section 4.3.2.

The data from the leg-worn sensor package were excluded after inspection of the recorded signals. The leg inertial data contained relatively little activity-specific information for the bench-based assembly tasks, apart from activities such as walking. Likewise, the leg microphone primarily recorded ambient noise and captured the assembly activities poorly. This thesis also did not use the inertial data of the non-dominant wrist and waist as model inputs. Integrating information across more than one body location, at the feature level or at the decision level, is instead noted as a direction for future work, addressed in Chapter 7.

Placement	**Modalities recorded**	**IMU as model input**	**Audio as model input**
Dominant wrist	inertial + audio	yes	yes
Non-dominant wrist	inertial + audio	no	no
Waist	inertial + audio	no	no
Leg	inertial + audio	no	no

Table 3.2: Body placements: data recorded during acquisition versus data used as model input in the final pipeline. Only the dominant-wrist inertial and acoustic streams reach the fusion models.

3.4 Recording Protocol

The same procedure was followed for each recording session. All devices were worn by the acting participant and fastened with straps to keep the sensor axes fixed relative to the body segment to which they belonged. Before recording started, all devices were checked to ensure that they were connected and transmitting data correctly. The participant then repeated the set of predefined assembly steps at the workbench while the second participant filmed the assembly sequence with a handheld phone. The host computer displayed the incoming packets from all devices, so that a disconnection or a sustained loss of packets could be noticed; no session required a restart.

Table 3.3 provides an overview of the recorded sessions. The session durations were calculated from the recorded inertial streams, and the number of two-second

windows was counted from the final dataset. Sessions 1–3 contributed 320, 343 and 604 windows, and sessions 4–6 contributed 164, 738 and 396 windows. In sessions 1–3, the distribution of classes follows the recording protocol. Some actions, such as screwdriving, were performed several times within each repetition, whereas others, such as placing a container, occurred only once. Sessions 4–6 were used to add data for the classes that were under-represented in this way. The resulting class distribution is discussed in Section 3.8.

Session	Type	Duration	Reps	Windows
1	Assembly sequence, all four placements	≈6.7 min	3	320
2	Assembly sequence	≈7.6 min	3	343
3	Assembly sequence, one invalid	≈16.9 min	4 (+1)	604
4	Activity-focused: carrying a container	≈12.1 min	–	164
5	Activity-focused: idle, walking, assembly steps, screwdriving	≈16.5 min	–	738
6	Activity-focused: idle, hammering, measuring, placing	≈8.3 min	–	396
Total		**≈68.1 min**		**2565**

Table 3.3: Data-acquisition sessions. "(+1)" denotes one further sequence marked invalid and set aside. Sessions 4–6 consist of longer bouts of individual activities instead of repetitions of the full sequence.

3.5 Ground-Truth Annotation from Synchronised Video

The sensor recordings provide motion and acoustic information but do not contain activity labels. The ground truth was therefore obtained from the recorded video, which provides a visual reference for determining which activity was being performed at a given point in time. The annotated video intervals were then transferred to the sensor recordings. This process requires the video and sensor data to be placed on a common timeline and the activities to be represented as defined time intervals.

3.5.1 Determination of the Audio-Video Offset

The video and sensor recordings were started independently, meaning that their time origins were not identical. Before the video annotations could be mapped to the sensor data, a temporal offset between the two recordings had to be determined.

The offset was calculated by finding an event that is clearly identifiable in both the video and the recorded audio. The audio from the sensor side was analysed

in Audacity [2] to find the corresponding transient, which was located within a millisecond. The same event, a spoken cue that the participant announced at the start and end of each assembly sequence, was then found in the video's audio track. The difference between the event times in the sensor recording and the video provides the required temporal offset.

For instance, the offset measured for one session is 3.857 seconds. This value is used for the synchronisation procedure to take out the same amount of data from the beginning of each sensor stream, so that the beginning of the sensor timeline becomes the zero point of the video. In each recording session, the offset was determined independently and applied to the synchronised streams used in this thesis. This step appears in the full preprocessing pipeline shown in Figure 3.3.

3.5.2 Interval Labels and Sample-Level Label Embedding

The activities were annotated directly from the video using BORIS [13], a tool for behavioural observation and event logging. Every annotation specifies a start time, an end time, and an activity label on the video timeline. The annotations were exported to a table with the columns `Label`, `Start`, and `End`.

These annotation intervals were then moved to the sensor data. For each inertial sample, its timestamp relative to the beginning of the recording was compared with the annotated intervals. If the timestamp was within an activity interval, the associated activity label was attributed to the sample. Samples that were not part of any of the annotated intervals were assigned a reserved `unlabeled` marker. This avoids assigning an activity label to transition periods or moments in which no specific task was being performed.

The assigned label was saved as a separate column along with the inertial measurements, and thus the inertial recording was transformed into a labelled time series. The audio stream was labelled using the same temporal reference. Since the audio and inertial data originate from the same device and were aligned to the video timeline, an audio segment corresponds to the activity represented by the inertial samples covering the same time interval. Consequently, transitions and untasked periods remain explicitly represented as `unlabeled`. Their treatment during the subsequent windowing process is described in Section 3.7.

3.6 Signal Conditioning

Before the data were divided into windows, the recorded streams were processed to compensate for transmission-related artefacts and differences in the device clocks. Next, the inertial and acoustic data were brought to fixed sampling rates to ensure consistent processing across all the recordings.

The conditioning process was repeated for each of the devices and yielded continuous, uniformly sampled, and temporally aligned inertial and acoustic streams. The entire pre-processing chain is shown in Figure 3.3.

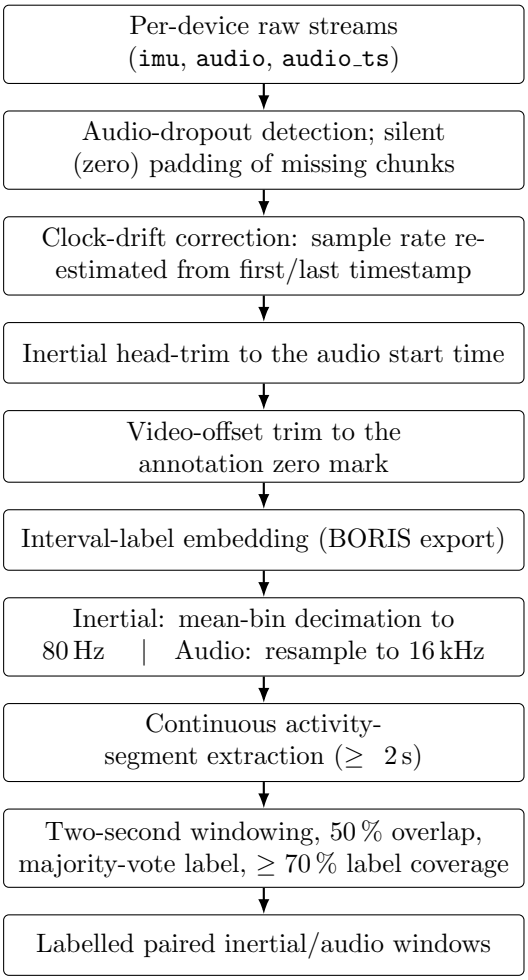


Figure 3.3: Signal-conditioning and windowing pipeline, applied per device. The inertial and acoustic branches are brought to fixed rates before both are cut on the same window boundaries.

3.6.1 Packet-Loss Detection and Audio-Dropout Reconstruction

Missing audio packets appear as gaps in the recorded waveform. Since each audio packet contains an arrival timestamp and a fixed number of samples, the expected time between consecutive packets can be determined. At a sampling rate of 16 kHz, for example, a packet containing 512 samples represents 32 ms of audio, so under normal conditions consecutive packets are expected to be separated by approximately 32 ms.

A larger time gap between two received packets indicates that one or more packets may have been lost. The number of missing packets is estimated from the observed time difference using

$$\mathrm{round}\left(\frac{\Delta t - T_{\mathrm{packet}}}{T_{\mathrm{packet}}}\right),$$

where Δt denotes the observed time difference between two consecutive received packets and T_{packet} denotes the expected packet interval. For the 16 kHz audio stream with 512 samples per packet, $T_{\mathrm{packet}} = 32\,\mathrm{ms}$. For example, if two received packets are separated by 96 ms, the equation gives

$$\mathrm{round}\left(\frac{96 - 32}{32}\right) = 2.$$

This means that two packets are assumed to be missing between the two received packets.

If a packet is detected as missing, a block of 512 zero-valued samples is inserted at the appropriate location in the waveform. This approach does not attempt to reconstruct the missing acoustic content. Instead, its purpose is to preserve the correct duration and temporal position of the surrounding signal so that subsequent time-frequency analysis is not affected by a shortened audio stream. The missing regions are therefore represented as silence. This is a conservative approach, as it introduces no artificial acoustic events, at the cost of a short period of artificially silent audio.

The fraction of reconstructed audio, computed from the raw per-device packet-arrival timestamps, is 0.15–0.40 % per session and about 0.33 % overall for the dominant wrist (Section 3.9); this is numerically identical to the packet-loss rate, since every missing packet is exactly 512 samples of silence.

The sequence numbers are used to detect the loss of packets in the inertial streams as well. In the dominant-wrist stream, the packet-loss rate was roughly 0.035 % overall (Section 3.9). As described in Section 3.6.2, the short gaps in the inertial data were then filled by linear interpolation on the resampled time grid.

3.6.2 IMU Decimation to 80 Hz

The inertial stream is delivered at the device native rate, measured at approximately 220–250 Hz across the sessions from the devices' microsecond timestamps. The firmware's own in-code comment labels the relevant configuration call "100 Hz",

which does not match the measured rate. The discrepancy has no effect on the modelling data, since decimation always targets a fixed 80 Hz regardless of the input rate. The stream is reduced to this uniform 80 Hz for modelling.

The reduction is performed by mean binning. The stream is divided into consecutive 12.5 ms bins, matching the desired sampling period at 80 Hz, and the inertial values in each bin are averaged to produce one output sample. If a bin contains no samples, for instance because of packet loss, its value is estimated by linear interpolation between the surrounding samples. The label column is decimated separately by taking the most frequent label in each bin. The resulting stream sits on an exact 80 Hz grid, which removes the small timing irregularities of the native stream.

Mean binning also provides a simple form of smoothing, although it is not used as a substitute for a specifically designed anti-aliasing filter. In particular, averaging within each bin attenuates signal components above the new Nyquist frequency of 40 Hz to some degree, but no filter with a defined passband or stopband was designed as part of this preprocessing step.

The rate of 80 Hz was selected based on the target platform and the anticipated nature of the activities to be captured. The sampling rate is intended to retain the frequency content relevant to the wrist movements involved in manual assembly, while substantially reducing the amount of inertial data and the resulting model input size [CITATION]. This reduces the inertial sampling rate by roughly a factor of three compared with the measured native rate.

3.6.3 Audio Resampling to a Strict 16 kHz Timeline

Following clock-drift correction, every audio stream is resampled from its effective sampling rate to a strict 16 kHz timeline using the band-limited resampler in librosa [24]. The resulting signal is stored as 16-bit PCM.

Using a fixed sampling rate ensures that the number of audio samples directly corresponds to a fixed duration. This is crucial for the windowing and feature-extraction stages that follow, because each example must span the same time interval and generate a consistent input representation. The sampling rate of 16 kHz is also sufficient to capture the frequency range considered relevant for the recorded assembly activities, which include impulsive and periodic sounds such as those generated when hammering and screwdriving. It is also consistent with the sampling rate used by the audio feature-extraction pipeline described in Chapter 4.

3.7 Window Generation and Labelling

After signal conditioning, the synchronised streams are first cut into continuous activity segments. Each segment is associated with one annotated activity interval and is only considered for further processing if it has a duration of at least two seconds. These segments are later split into equal-size windows for classification.

Each window has a duration of two seconds, with a 50 % overlap between consecutive windows, so a new window starts every one second. Overlapping windows are common in activity recognition because they increase the number of training examples and reduce the chance that a short activity is sliced at a window boundary [8]; the selected two-second window length and 50 % overlap are within the range commonly used in wearable HAR [3]. For the selected sampling rates, each two-second window contains 160 inertial samples at 80 Hz and 32 000 audio samples at 16 kHz. Both modalities are segmented using the same temporal boundaries on the shared timeline, so each inertial window has a corresponding audio window of the same span; the two together form one paired multimodal example.

The activity label of each window is decided using majority vote. The vote counts only samples that carry a valid activity label; samples tagged as `unlabeled` (Section 3.5.2) are not included. The activity that appears most often among the labelled samples is assigned to the window.

To prevent transition periods from becoming part of the training data, a minimum label-coverage threshold of 70 % is applied. A window is kept only if at least 70 % of its samples have a valid activity label. A window with a high proportion of `unlabeled` samples is not assigned to an activity class but is discarded instead, usually at transitions between activities. This method keeps windows with many transitions out of the training data while preserving windows that may have some activity-boundary overlap. Figure 3.4 shows the full window generation and labelling process.

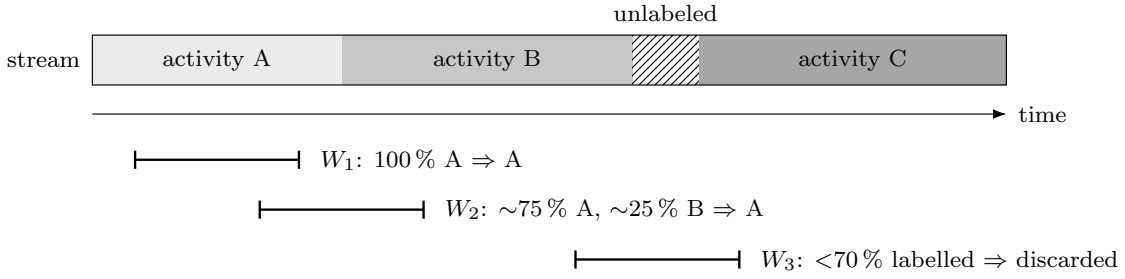


Figure 3.4: Window generation and majority-vote labelling. Two-second windows advance in one-second steps (W_1, W_2, W_3 shown stacked for clarity). A window takes the majority activity among its labelled samples; a window whose labelled coverage falls below 70 %, such as one spanning the transition region, is discarded.

Windowing was carried out separately for each body placement and session, producing one set of labelled inertial and audio windows per placement, organised by class.

3.8 Dataset Statistics, Class Distribution, and Imbalance

The dataset used for modelling is the dominant-wrist subset. It has 2565 paired inertial and audio windows for the nine activity classes outlined in Table 3.1. The data come from two participants and six recording sessions and represent roughly 43 minutes of windowed activity out of the 68 minutes of recorded data. The remaining time is not part of the modelling dataset because of excluded classes, the invalid sequence, unannotated periods between the recorded activities and windows removed around activity transitions.

Each inertial window contains a 160×6 array of accelerometer and gyroscope measurements sampled at 80 Hz. Each corresponding audio window contains 32 000 mono samples at 16 kHz. Table 3.4 summarises the composition of the dataset, and Figure 3.5 shows the distribution of windows across the activity classes.

Property	**Value**
Paired inertial/audio windows	2565
Activity classes	9
Participants	2
Recording sessions	6
Windowed activity	≈43 min
Window length	2 s
Window stride	1 s (50 % overlap)
Minimum label coverage	70 %
Inertial window	160×6 at 80 Hz
Audio window	32 000 samples, mono, 16 kHz
Inertial channels	acc_x, acc_y, acc_z, gyro_x, gyro_y, gyro_z
Additional audio windows (standalone audio model)	2224 (non-dominant wrist and waist, sessions 4–6)

Table 3.4: Composition of the dominant-wrist modelling dataset.

The class distribution is imbalanced. The smallest class, `place_container`, contains 74 windows, roughly one eighth of the largest class, `screw_tightening`, with 596 windows. The imbalance results mainly from the design of the assembly task, in which screwdriving is repeated many times per sequence while other activities, such as placing a container, occur only once. It is not corrected during preprocessing. How it is handled in training and evaluation is detailed in Chapter 4, and its impact on the findings in Chapter 6.

The number of windows discarded because of the 70 % label-coverage requirement was not recorded for sessions 1–3. For sessions 4–6, 911 windows of the dominant wrist were discarded, most of them in periods between the recorded activities that were not annotated.

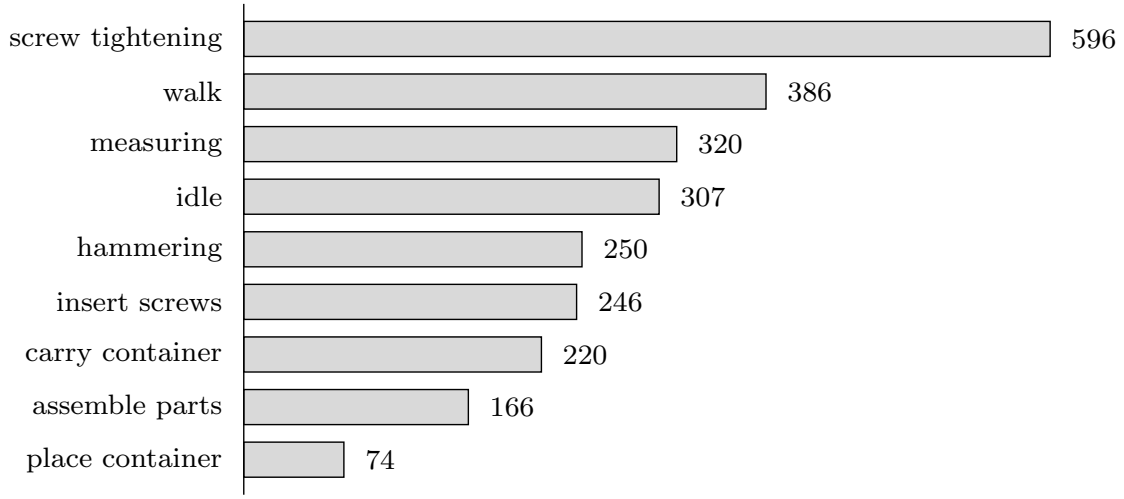


Figure 3.5: Number of two-second windows per activity class in the dominant-wrist dataset ($n = 2565$). `screw_tightening` is the largest class with about 23 % of the windows, while `place_container` is the smallest with 74 windows.

3.9 Data-Quality Assessment and Known Artefacts

There are a number of characteristics of the data that are important to consider. These characteristics do not prevent the dataset from being used for the experiments in this thesis, but they define the limits of what can reasonably be concluded from those experiments. Table 3.5 summarises the primary data-quality indicators that could be established.

Packet loss and audio reconstruction. Wireless UDP transmission loses packets. Inertial loss on the dominant-wrist stream is measured at about 0.035 % overall (Section 3.6.1). Audio loss, computed from the raw per-device packet-arrival timestamps, is higher: 0.15–0.40 % per session and about 0.33 % overall for the dominant wrist, roughly an order of magnitude above the inertial loss rate. Because every missing audio packet is reconstructed as exactly 512 samples of silence (Section 3.6.1), the reconstructed-audio fraction is numerically the same figure.

Synchronisation uncertainty. The alignment of the sensor recordings with the activity annotations is based on a manually identified synchronisation event for each session, followed by a fixed offset applied to the entire session (as described in Section 3.5.1). There is no quantification of the residual error caused by manual event locating, and no characterisation of the gradual clock drift between the recordings. Majority voting applied over two-second windows and the 70 % threshold for label coverage minimise the risk of a small window alignment error affecting the label assigned to a particular window. However, this kind of error can still change the effective boundaries of an activity.

Scope of the dataset. The dataset was collected from two participants performing a scripted assembly task in a single laboratory environment. The final

Indicator	Value	Notes
Inertial packet loss (dominant wrist)	≈0.035 % overall	From sequence-number gaps. Below the 1 % design target.
Audio packet loss (dominant wrist)	0.15–0.40 % per session; ≈0.33 % overall	From `audio_ts` packet-arrival timestamp gaps, computed from the raw per-device logs. Reconstructed audio (inserted silence) is the same fraction, since every missing packet is exactly 512 samples.
Synchronisation basis	one manual event per session	Residual reading error and slow clock drift over a session are not characterised.
Camera	handheld, moving	Framing varied, but no hand action was occluded during annotation.
Discarded low-coverage windows	not recorded (sessions 1–3); 911 (sessions 4–6)	See Section 3.8.

Table 3.5: Data-quality indicators for the dataset.

modelling data represent approximately 43 minutes of windowed activity, with one class accounting for almost a quarter of all windows. The dataset is suitable for the comparative analysis of multimodal fusion methods and on-device feasibility undertaken in this thesis. Its size and limited variation do not make it suitable for assessing how the resulting models would perform for different participants, assembly tasks, or environments. These limitations are addressed in more detail in Chapter 6, which also discusses their effect on the experimental results.

3.10 Chapter Summary

This chapter described the construction of the multimodal dataset. Inertial and acoustic data were streamed over Wi-Fi from up to four M5StickC Plus2 devices worn at the wrists, the waist and, in sessions 1–3, the leg, using a UDP-based firmware that timestamps and sequence-numbers every packet so that the streams can be realigned after transmission. Two participants recorded six sessions in the Fraunhofer IPA Future Work Lab, alternating between performing the task and filming it on a handheld phone. Sessions 1–3 followed the full assembly sequence, and sessions 4–6 focused on longer bouts of individual activities to add data for under-represented classes. Activity intervals were coded from the video in BORIS and transferred onto the sensor timeline after the sensor-to-video offset was fixed from a shared event located in Audacity. Each device stream was then corrected for audio dropouts and clock drift, decimated to 80 Hz for the inertial channels and resampled to 16 kHz for audio, and segmented into overlapping two-second windows labelled by majority vote subject to a 70 % coverage threshold. The modelling dataset used in the remainder of this thesis is the dominant-wrist subset, with 2565 paired inertial and audio windows over nine activity classes, a class imbalance led

by screwdriving and the data-quality limitations set out in Section 3.9. Additional audio from the non-dominant wrist and the waist of sessions 4–6 is used only by the standalone audio model. This dataset is the common input to all of the models and experiments described in the following chapters.

4 Model Design, Training and Deployment

This chapter describes the models used for activity recognition and the procedure used to train and evaluate them. It begins with an overview of the experimental design and its relation to the research questions. It then covers the input representations derived from the inertial and acoustic windows of Chapter 3, the single-modality models, the three fusion strategies, and a classical baseline using hand-crafted features. It closes with the training procedure, the deployment of selected models on the wearable device and the evaluation protocol that all models share. The results obtained with these models are presented in Chapter 5.

4.1 Overview of the Experimental Design

The experiments compare models that differ in one respect at a time. All models are trained and evaluated on the same windows and the same train/validation split, and they use the same input normalisation and the same training settings. Differences in accuracy can therefore be attributed to the model design and not to differences in the data or in the training procedure.

The models fall into four groups, each linked to one or more of the research questions formulated in Chapter 1:

- **Single-modality models** use either inertial or acoustic data alone. They form the reference point against which the benefit of adding audio is measured (RQ1, RQ3).
- **Fusion models** combine both modalities at different stages of processing. They are combined at the input level (early fusion), at the level of learned representations (embedding-level fusion), or at the level of class predictions (decision-level fusion). Their comparison addresses RQ2.
- **A classical baseline** uses conventional machine learning without deep neural networks. Simple statistical features are computed by hand from each window and classified with a random forest. Comparing the neural networks with this baseline shows how much is gained by learning the features from the data instead of defining them in advance (RQ3).
- **Reduced model variants** with fewer parameters were designed with the memory and computational limits of the target device in mind. They connect

the offline comparison to the deployment on the device described in Section 4.8 (RQ4).

Group	**Model**	**Inputs**	**Fusion stage**
Single modality	IMU CNN	inertial	–
Single modality	Audio CNN (GAP, GRU and hybrid pooling)	acoustic	–
Early fusion	Channel concatenation (standard and small)	inertial + acoustic	input
Early fusion	Unified spectrogram	inertial + acoustic	input
Embedding-level fusion	Frozen backbones with a trained fusion head (standard and small)	inertial + acoustic	representation
Decision-level fusion	Weighted average with global or per-class weights (standard and small)	inertial + acoustic	prediction
Classical baseline	Random forest on hand-crafted features	inertial + acoustic	feature

Table 4.1: Overview of the evaluated models.

The choice of model types follows from two requirements of this thesis. The models must recognise activities from short windows of sensor data, and the best of them must later run on a microcontroller with very limited memory and computing power. Convolutional neural networks (CNNs) meet both requirements. They learn characteristic local patterns directly from the signal, such as the shape of a hammer strike in the acceleration or the frequency content of a screwdriver in the spectrogram, and they need comparatively few parameters because the same small filters are applied across the whole window. For these reasons, CNNs are among the most widely used architectures in wearable activity recognition and in embedded audio classification [18, 35, 40] [CITATION: keyword spotting on microcontrollers]. Recurrent layers can capture the temporal order of a signal more explicitly. Small recurrent models can run on microcontrollers [40], but they process the time steps one after the other and use operations that are less well supported there, as discussed in Section 2.6. They are therefore included only as an optional pooling stage of the audio model, so that their benefit can be measured against their cost. The three fusion strategies cover the three stages at which modalities are commonly combined in multimodal learning [1]. The random forest, finally, is a standard reference method in activity recognition that performs well on small datasets and requires little tuning [9, 16].

All experiments use the dominant-wrist dataset described in Chapter 3, with 2565 paired windows across nine activity classes. The only exception is the standalone

audio model, which additionally uses audio windows from the non-dominant wrist and the waist. The reason for this is given in Section 4.3.2.

4.2 Input Representations

4.2.1 Inertial Windows

Each inertial window holds 160 samples of the six channels (three-axis acceleration and three-axis angular velocity) recorded at 80 Hz. Before entering a model, the window is resampled to 200 time steps by linear interpolation along the time axis, so that all inertial models take inputs of size 200×6. The target length of 200 was chosen to match the time resolution of the audio spectrogram, which, as described in Section 4.2.2, has 198 frames per window. This allows the early fusion model presented in Section 4.4.1 to place both modalities on one time axis without coarsening the audio representation. Using the same length in all inertial models also keeps their inputs identical across the experiments. Missing or non-finite values, which can remain at the very edges of a recording after decimation, are replaced by zero.

Accelerometer and gyroscope channels have different physical units and value ranges. Each channel is therefore standardised to zero mean and unit variance. The mean and standard deviation of each channel are computed on the training split only and then applied unchanged to the validation data, so that no statistics of the validation set enter the training process. As described in Section 4.8, the same statistics are later embedded in the device firmware and applied to the live sensor data.

4.2.2 Audio Log-Mel Spectrograms

Raw audio windows of 32 000 samples are too long to be used directly as input to a small model. Each window is therefore converted into a log-mel spectrogram, a time-frequency representation that is widely used for sound classification [CITATION]. The conversion consists of three steps. First, a short-time Fourier transform (STFT) splits the signal into overlapping frames of 25 ms (400 samples), shifted by 10 ms (160 samples), and computes the frequency spectrum of each frame with an FFT length of 512. Second, the magnitude spectrum of each frame is projected onto 40 frequency bands between 20 Hz and 8 kHz. The bands are spaced according to the mel scale, which resolves low frequencies more finely than high ones, similar to human hearing [CITATION]. Third, the natural logarithm of the band energies is taken. A small constant of 10^{-6} is added beforehand so that silent bands do not produce the logarithm of zero.

A two-second window produces a spectrogram of 198 frames $\times$ 40 mel bands. This reduces the input size by roughly a factor of four compared with the raw waveform, while keeping the spectral and temporal structure that distinguishes, for example, the continuous whine of a powered screwdriver from the short, broadband impacts of

hammering. As with the inertial data, the spectrogram is standardised per mel band using the mean and standard deviation of the training split. Figure 4.1 summarises the feature extraction, and Table 4.2 lists the resulting input representations.

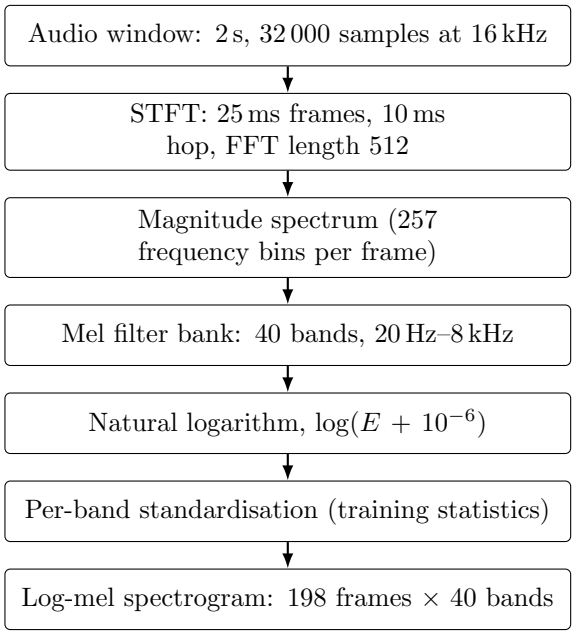


Figure 4.1: Conversion of a two-second audio window into the log-mel spectrogram used as input to the acoustic models.

Modality	**Raw window**	**Model input**	**Normalisation**
Inertial	160×6 at 80 Hz	200×6 (linear resampling)	per channel, training statistics
Audio	32 000 samples at 16 kHz	198×40 log-mel	per mel band, training statistics

Table 4.2: Input representations of the two modalities.

4.3 Single-Modality Models

4.3.1 Inertial Model

The inertial model is a one-dimensional convolutional neural network (1D CNN) that convolves along the time axis of the window. 1D CNNs are a common choice for wearable activity recognition because they learn local motion patterns directly from the raw signal and remain small enough for embedded use [35, 1, 40].

The network has two convolutional blocks. The first block applies 64 filters with a kernel size of 5, followed by batch normalisation and max pooling with a factor

of 2. The second block applies 128 filters with a kernel size of 3, followed by batch normalisation. Global average pooling then reduces the temporal dimension to a single feature vector. This vector passes through a dense layer with 64 units and through dropout with a rate of 0.3, before a softmax output layer with one unit per class. The output of the 64-unit layer also serves as the embedding for the embedding-level fusion described in Section 4.4.2.

4.3.2 Audio Model and Temporal Pooling Variants

The audio model treats the log-mel spectrogram as a single-channel image and processes it with a two-dimensional CNN, as is common for sound classification [16]. It has two convolutional blocks with 32 and 64 filters of size 3×3, each followed by batch normalisation and 2×2 max pooling. The resulting feature map has a size of $49 \times 10 \times 64$ (time $\times$ frequency $\times$ channels).

How this feature map is reduced to a single vector determines which properties of the sound the classifier can use. Three variants were therefore evaluated, which are shown in Figure 4.2.

- **Global average pooling (GAP).** The feature map is averaged over both time and frequency [CITATION]. This is the standard approach and gives a compact model. However, averaging over time removes the information about how the sound develops within the window. Two activities with a similar average spectrum but a different temporal pattern, such as the regular strikes of a hammer and the steady sound of a screwdriver, can then become hard to separate.

- **Recurrent pooling (GRU).** The feature map is reshaped into a sequence of 49 time steps with 640 features each and passed through a bidirectional gated recurrent unit (GRU) with 32 units per direction [CITATION]. The GRU summarises the sequence while keeping its temporal order, so the model can use rhythm and envelope information.

- **Hybrid pooling.** The GAP vector and the GRU output are computed in parallel and concatenated. The GAP vector describes the average timbre of the window, and the GRU output describes its temporal structure. The classifier can then rely on whichever information is useful for a given class.

In all three variants, the pooled vector passes through a dense layer with 64 units and dropout with a rate of 0.4, followed by the softmax output.

Because audio content depends less on where the sensor is worn than inertial data does, the standalone audio model is trained on audio windows from the dominant wrist, the non-dominant wrist and the waist. This roughly triples the amount of acoustic training data. The pooled audio is used only for the standalone audio model. All fusion models use dominant-wrist audio exclusively, so that every audio window is paired with the inertial window from the same device and time interval.

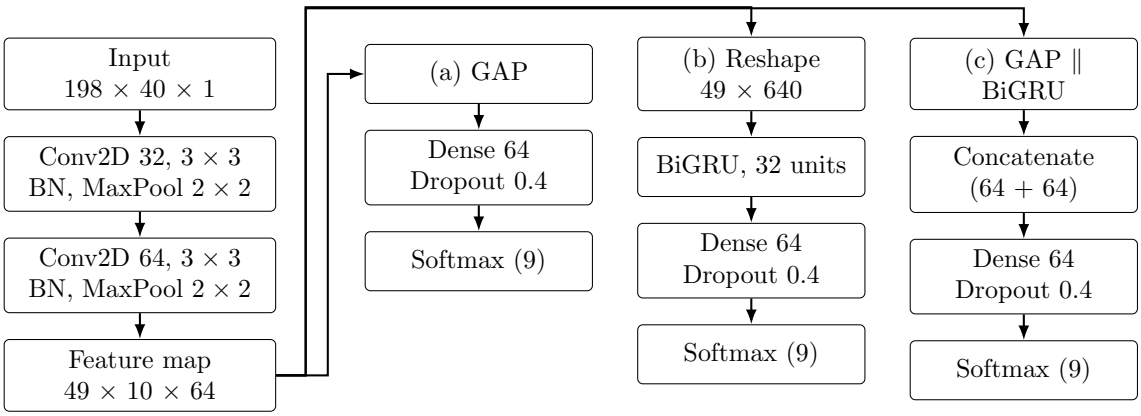


Figure 4.2: Audio model with a shared convolutional stem and the three pooling variants: (a) global average pooling, (b) recurrent pooling with a bidirectional GRU, and (c) hybrid pooling, which concatenates both summaries.

The pooling variants are compared on the standalone audio model. Within the fusion models, the acoustic component uses the GAP variant. With 23 945 parameters it is about six times smaller than the recurrent variants, and it avoids recurrent layers, whose sequential computation is costly on the target microcontroller. This keeps all fusion models within reach of the deployment described in Section 4.8. The performance of the three variants is reported in Chapter 5. Table 4.3 summarises both single-modality architectures.

Layer	**IMU CNN**	**Audio CNN**
Input	200×6	$198 \times 40 \times 1$
Block 1	Conv1D 64, kernel 5, BN, MaxPool 2	Conv2D 32, 3×3, BN, MaxPool 2×2
Block 2	Conv1D 128, kernel 3, BN	Conv2D 64, 3×3, BN, MaxPool 2×2
Pooling	Global average	GAP / BiGRU(32) / GAP + BiGRU
Head	Dense 64, Dropout 0.3	Dense 64, Dropout 0.4
Output	Softmax (9 classes)	Softmax (9 classes)
Parameters	36 297	23 945 (GAP) / 153 353 (GRU) / 157 449 (hybrid)

Table 4.3: Architecture of the single-modality models. All convolutional and dense layers except the output use ReLU activations.

4.4 Fusion Architectures

The three fusion strategies differ in the point along the processing chain at which the two modalities are combined [1]. Figure 4.3 gives an overview.

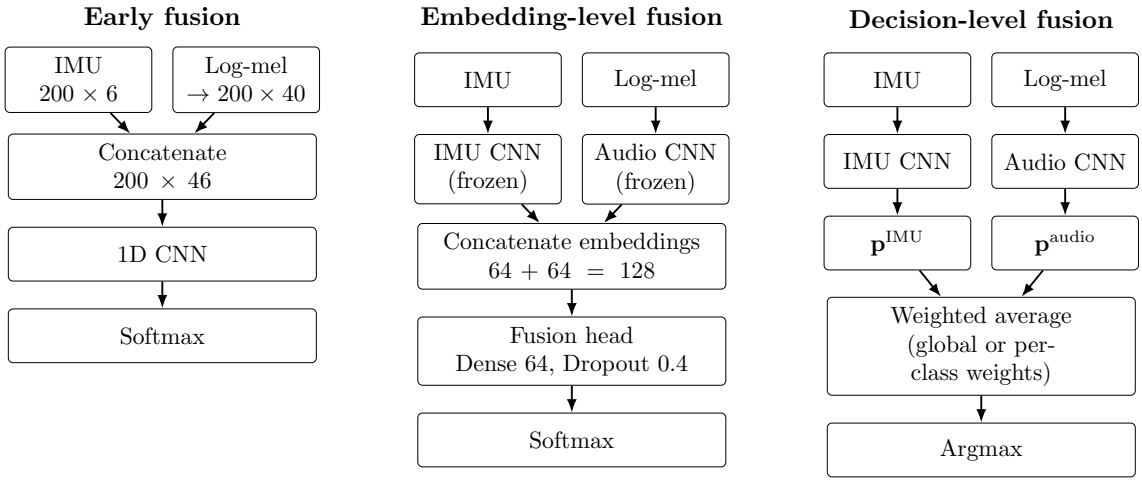


Figure 4.3: The three fusion strategies. Early fusion combines the raw inputs before learning, embedding-level fusion combines learned 64-dimensional representations of two frozen backbones, and decision-level fusion combines the class probabilities of two independently trained models.

4.4.1 Early Fusion

Early fusion combines the two modalities before any learning takes place, so that a single network learns from both signals jointly. Two forms were implemented.

Channel concatenation. The log-mel spectrogram (198×40) is resampled along its time axis to 200 frames, matching the length of the inertial window. The two inputs are then concatenated along the feature axis, giving a 200×46 input with 6 inertial and 40 acoustic channels. Each of the 46 channels is standardised with its own training statistics. This step is particularly important here, because the channels of the fused input have very different value ranges. In the recorded data, the acceleration lies roughly between -1.5 and $+1.5\,\mathrm{g}$ and the angular velocity between -200 and $+200\,^\circ/\mathrm{s}$, while the log-mel values lie roughly between -14 and $+3$. A convolutional filter sums weighted values from all channels. Without standardisation, the gyroscope channels would therefore dominate these sums simply because of their larger numbers, and the network would first have to learn to compensate for the different units before it could learn from the content of the signals. After standardisation, every channel has zero mean and unit variance and starts with the same influence on the result. The fused input is then processed by a 1D CNN with the same structure as the inertial model described in Section 4.3.1.

Unified spectrogram. Here the inertial data are also converted into spectrograms, so that both modalities share one image-like representation. A short-time Fourier transform with a frame length of 20 samples, a hop of 2 samples and an FFT length of 32 is applied to each of the six inertial axes. The log magnitudes are resized to the 198×40 grid of the audio spectrogram by bilinear interpolation. The six inertial spectrograms and the audio spectrogram are stacked as the seven channels of a single input and processed by the 2D audio CNN with global average pooling.

4.4.2 Embedding-Level Fusion

Embedding-level fusion first trains the inertial and the acoustic model separately, as described in Section 4.3. Their weights are then frozen, and for each window the outputs of the 64-unit dense layer are taken as embeddings. The two 64-dimensional embeddings are concatenated into a 128-dimensional vector, which is the input to a small fusion head. The head consists of a dense layer with 64 units, dropout with a rate of 0.4 and a softmax output, with 8841 parameters in total. Only the fusion head is trained in this second stage. It can learn how the two modalities relate to each other without changing the feature extraction that each backbone has already learned.

4.4.3 Decision-Level Fusion

Decision-level fusion combines the predictions of the two independently trained models [1]. Both models end in a softmax layer, which outputs one value between 0 and 1 for each of the nine classes, with all nine values summing to 1. These values can be read as the confidence of the model that the window belongs to the respective class. For a window x, let $p_c^{\mathrm{IMU}}(x)$ denote the probability that the inertial model assigns to class c, and $p_c^{\mathrm{audio}}(x)$ the probability that the acoustic model assigns to the same class. The fused score of class c is the weighted average of the two,

$$p_c(x) = w \cdot p_c^{\mathrm{IMU}}(x) + (1 - w) \cdot p_c^{\mathrm{audio}}(x), \qquad w \in [0, 1]. \tag{4.1}$$

The weight w determines how much the inertial model is trusted relative to the acoustic model. With $w = 1$, only the inertial model is used, with $w = 0$, only the acoustic model is used, and with $w = 0.5$, both count equally. The fused score is computed for every class, and the class with the highest score is the prediction. The weight is selected by trying the 21 values $0, 0.05, \ldots, 1$ and keeping the one that gives the highest accuracy on the validation data. Compared with a trained fusion head, this method has only a single free parameter and is therefore less prone to overfitting on a small dataset.

A single weight, however, assumes that the relative reliability of the two modalities is the same for every class. There is little reason to expect this for the present activity set, because the activities differ strongly in how much characteristic motion

and how much characteristic sound they produce. For some classes, the inertial model is therefore likely to be the more reliable source, and for others the acoustic model. Which modality is more reliable for which class is not fixed in advance but determined from the data. The method was therefore extended to one weight per class. With $C = 9$ classes, this gives nine weights $w_1, \ldots, w_9$, and the fused score of class c becomes

$$p_c(x) = w_c \cdot p_c^{\text{IMU}}(x) + (1 - w_c) \cdot p_c^{\text{audio}}(x). \tag{4.2}$$

The only difference to Equation 4.1 is that the weight now carries the index c. When the fused model computes the score of a particular class, it uses the weight of that class, and it can therefore rely on whichever sensor is more informative for that class. A value of w_c close to 1 means that the score of class c is determined mainly by the inertial model, and a value close to 0 means that it is determined mainly by the acoustic model.

A short example illustrates the effect. Consider a window of the class `measuring`, for which the two models produce the probabilities shown in Table 4.4. For readability, only three classes are shown. The inertial model recognises the activity, whereas the acoustic model hears almost silence and leans towards `idle`. With a global weight of $w = 0.5$, the correct class wins only by a small margin. If the weight of `measuring` is raised to $w_{\text{measuring}} = 0.9$, its score is taken almost entirely from the more reliable inertial model, and the margin increases clearly. The score of `idle` does not change, because its own weight has not changed.

	`measuring`	`idle`	`hammering`
$\mathbf{p}^{\text{IMU}}$	0.70	0.25	0.05
$\mathbf{p}^{\text{audio}}$	0.20	0.60	0.20
Global weight, $w = 0.5$	**0.45**	0.425	0.125
Per-class weight, $w_{\text{measuring}} = 0.9$, others 0.5	**0.65**	0.425	0.125

Table 4.4: Illustrative example of global and per-class decision-level fusion for a `measuring` window. The values are chosen for illustration and are not taken from the experiments.

It remains to find suitable values for the nine weights. For the single weight of Equation 4.1, all 21 candidate values could simply be tried. For nine weights, trying every combination would require 21^9, or about 800 billion, evaluations, which is not practical. The weights are therefore adjusted one at a time, while all others are kept fixed. This simple strategy is known as coordinate ascent [CITATION]. The name refers to the fact that only one coordinate of the weight vector, that is, one class weight, is changed in each step, and that a change is kept only if it raises the accuracy. The search proceeds as follows.

1. All nine weights start at the best single weight found for Equation 4.1. The search therefore begins with the result of global fusion and can only improve on it.

2. The weight of the first class is set in turn to each of the 21 values $0, 0.05, \ldots, 1$, while the other eight weights stay unchanged. For each value, all validation windows are classified with Equation 4.2, and the accuracy is computed. The value with the highest accuracy is kept, provided that it is better than the accuracy before this step.

3. Step 2 is repeated for the second class, then for the third class, and so on, until each of the nine weights has been adjusted once.

4. Since a change in one weight can make a different value of an earlier weight slightly better, steps 2 and 3 are carried out a second time.

In total, the search therefore evaluates $2 \times 9 \times 21 = 378$ weight settings, which takes less than a second. A second round is sufficient because the weights hardly influence each other. Changing the weight of one class only affects windows for which this class and another class received almost the same score. For most windows, one class is clearly ahead, and changing a single weight does not alter the prediction. The resulting weights show, for each class, how much the fused model relies on the inertial and how much on the acoustic model. They are therefore also used to interpret the results in Chapter 5. Figure 4.4 summarises the procedure.

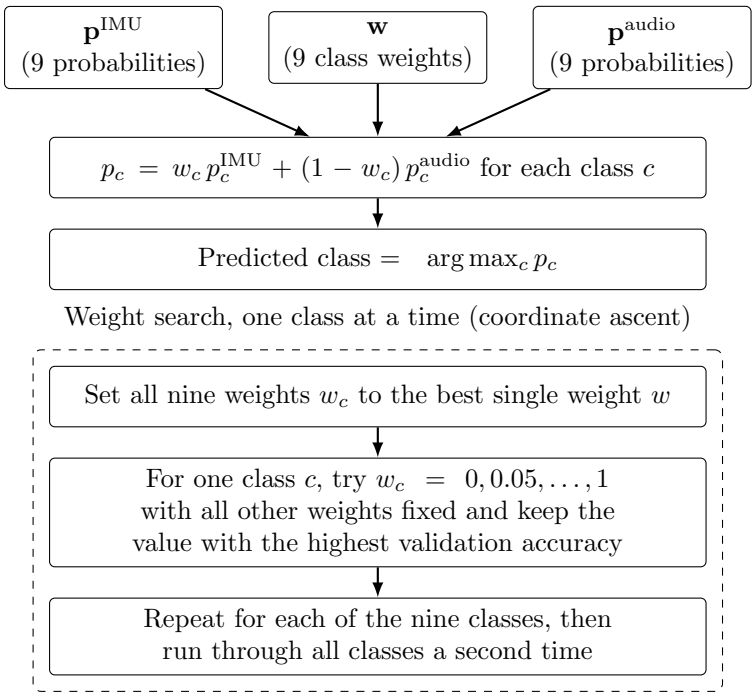


Figure 4.4: Decision-level fusion with per-class weights (top) and the coordinate-ascent search used to determine the weights on the validation data (bottom).

4.5 Classical Baseline

To place the neural networks in context, a classical machine-learning approach was also evaluated. In contrast to the deep learning models, which learn their features from the data during training, this approach relies on hand-crafted features, that is, statistics whose definition is fixed in advance. For each of the six inertial channels, the mean, standard deviation, minimum, maximum, root mean square and zero-crossing rate are computed, which gives 36 inertial features. For each of the 40 mel bands of the log-mel spectrogram, the mean and standard deviation over time are computed, which gives 80 acoustic features. As listed in Table 4.5, the two feature sets are concatenated into a 116-dimensional vector, so the baseline also combines both modalities at the feature level.

A random forest with 300 trees is trained on these vectors, with balanced class weights to account for the class imbalance [CITATION]. Random forests need no feature scaling and give robust results on small tabular datasets, which makes them a common reference point in activity recognition [9, 16].

Modality	**Features**	**Count**
Inertial	mean, standard deviation, minimum, maximum, RMS and zero-crossing rate for each of the 6 channels	36
Audio	mean and standard deviation over time for each of the 40 mel bands	80
Total		**116**

Table 4.5: Hand-crafted features of the classical baseline.

4.6 Deployment-Oriented Model Variants

The models described so far were designed for offline evaluation. On the target microcontroller, however, inference time and memory usage are strict limits. The TensorFlow Lite Micro build for this platform executes convolutions with unaccelerated reference kernels, so their cost grows directly with the number of filters and with the size of the feature maps. This is explained in more detail in Section 4.8.1. Reduced variants were therefore created that keep the overall architecture but use fewer filters.

- **Small early fusion model.** The structure of the channel-concatenation model with 32 and 64 instead of 64 and 128 filters. This reduces the number of multiply-accumulate operations, and with it the inference time, by roughly a factor of four.
- **Small inertial backbone.** A 1D CNN with 32 and 64 filters.

- **Small audio backbone.** A 2D CNN with 16 and 32 filters. Its convolutions use strides of 4 and 2 instead of separate pooling layers, so the downsampling happens within the convolution itself. As a result, a feature map at the full 198×40 resolution never has to be held in memory, which would otherwise take up a large part of the device's working memory.

The small backbones are used for the small embedding-level and decision-level fusion models. To reduce overfitting, three regularisation measures were added to them [CITATION]. L2 weight decay with a factor of 10^{-4} is applied to all learned layers, the dropout rates are raised to 0.4 and 0.5, and the loss function uses label smoothing of 0.1.

In addition, the training data of the small backbones are augmented. One augmented copy is generated for each training window, so that the training set doubles in size. The validation data are never augmented.

- **Inertial windows** receive a small random three-dimensional rotation of up to $\pm 15°$, applied jointly to the accelerometer and gyroscope axes to mimic small changes in how the device sits on the wrist, together with per-channel scaling (factor 0.9–1.1), additive noise (3 % of the channel's standard deviation) and a mild time warp (speed factor 0.9–1.1) [CITATION].

- **Spectrograms** are shifted in time by up to ± 10 frames, with the frames that leave one end of the window re-entering at the other. In addition, random parts of the spectrogram are masked, following the SpecAugment method originally proposed for speech recognition [CITATION]. Two blocks of up to 20 consecutive frames and two blocks of up to 6 adjacent mel bands are overwritten with the mean value of the spectrogram. Because the hidden parts change from copy to copy, the model cannot depend on a single short moment or a single frequency range and has to use the information of the whole spectrogram.

Table 4.6 compares the standard and the small variants.

4.7 Training Procedure

4.7.1 Software and Hardware Environment

All models were implemented in Python 3.10 with TensorFlow 2.10 and Keras [CITATION]. Training ran on an NVIDIA GeForce RTX 4070 Laptop GPU through the DirectML backend for Windows. This backend supports TensorFlow only up to version 2.10, which fixed the framework version for the whole project. Two parts of the pipeline could not run on the GPU with this backend and had to be adapted. The short-time Fourier transform used for the spectrograms was moved to the CPU, and the fused recurrent kernel of the GRU layer was replaced by an unrolled implementation. Both changes affect only where and how the computation is executed, not its result. Data preparation used NumPy, pandas and librosa [24], and the classical baseline used scikit-learn [CITATION].

Model	Standard	Small
Early fusion (concatenation)	Conv1D 64 / 128 49 097 parameters	Conv1D 32 / 64 18 729 parameters
IMU backbone	Conv1D 64 / 128 36 297 parameters	Conv1D 32 / 64, L2, dropout 0.4 12 329 parameters
Audio backbone	Conv2D 32 / 64 with max pooling 23 945 parameters	Conv2D 16 / 32 with strides 4 / 2, L2, dropout 0.5 7 689 parameters
Label smoothing	–	0.1 (backbones)
Augmentation	–	one augmented copy per training window (backbones)

Table 4.6: Standard and deployment-oriented (small) model variants. Parameter counts refer to the nine-class output layer.

4.7.2 Training Settings

All neural networks were trained with the settings listed in Table 4.7. The class imbalance described in Section 3.8 is handled through class weights in the loss function. The weight of each class is inversely proportional to its frequency in the training split, so that errors on rare classes count more than errors on frequent ones [CITATION]. The training data themselves are left unchanged, without oversampling or undersampling.

Setting	Value
Optimiser	Adam [CITATION]
Learning rate	10^{-3}, constant
Batch size	32
Loss	Categorical cross-entropy (label smoothing 0.1 for the small backbones)
Class weighting	Inverse class frequency (“balanced”)
Epochs	150 for backbones and early-fusion models, at most 80 for the fusion head
Model selection	Weights of the epoch with the highest validation accuracy
Fusion head	Early stopping (patience 15), learning rate halved on plateau (patience 6, minimum 10^{-5})

Table 4.7: Training settings shared by all neural network models.

The backbones and the early-fusion models are trained for a fixed 150 epochs at a constant learning rate. On a small validation set, the validation accuracy can

fluctuate noticeably from one epoch to the next, so the weights at the end of training are not necessarily the best ones. After training, each model is therefore set back to the weights of the epoch with the highest validation accuracy.

In the Keras version used here, the standard early-stopping callback restores the best weights only when it actually stops training early. If training runs the full number of epochs, the weights of the last epoch are kept without any warning. A small custom callback was therefore written that stores the best weights during training and restores them at the end. The effect of this choice is examined in Chapter 5. Because the best epoch is selected on the validation data, the procedure introduces a slight optimistic bias into the reported results, a point taken up in the discussion of limitations in Chapter 6.

4.8 Deployment on the Wearable Device

The models described so far were trained and evaluated on a desktop computer. To answer RQ4, selected models were also brought onto the M5StickC Plus2, the same device that recorded the training data. Two conditions must hold for this to succeed. The model has to fit into the memory of the microcontroller and run fast enough for continuous use. In addition, the device has to transform the live sensor signals in exactly the same way as the training pipeline did, because even a correct model produces wrong predictions if its input differs from what it saw during training. This section describes how both conditions were addressed. The measured memory use, inference time and on-device accuracy are reported in Chapter 5.

4.8.1 Target Platform and Software Stack

As described in Section 3.1, the device is built around the ESP32-PICO-V3-02, a dual-core processor clocked at up to 240 MHz with 520 KB of on-chip SRAM. Only part of this memory is available to the application, because the Arduino framework and the device libraries reserve their own share. The firmware was developed with PlatformIO and the Arduino framework. Neural network inference uses TensorFlow Lite for Microcontrollers [11], a version of TensorFlow Lite designed for devices without an operating system and without dynamic memory allocation. The FFT for the audio features is computed with the arduinoFFT library [CITATION].

The TensorFlow Lite Micro port available for this platform does not include the optimised neural network kernels that the chip manufacturer provides for newer processors. All layers are therefore executed with the generic reference implementations. This is the reason why the inference time grows directly with the number of filters and the size of the feature maps, and why the reduced model variants described in Section 4.6 were developed.

Three firmware variants were implemented, one for each fusion strategy. They share the same capture and preprocessing code but differ in the number of networks they run, as summarised in Table 4.8. Each variant is a separate PlatformIO

build environment, so that the data-collection firmware of Chapter 3 and the three inference variants can be flashed independently of each other.

Variant	**Networks on the device**	**Network inputs**	**Tensor arena**
Early fusion (small)	one fused network	200×46	40 KB
Embedding-level fusion (small)	IMU backbone, audio backbone and fusion head	200×6, 198×40 and 128	20, 20 and 3 KB
Decision-level fusion (small)	IMU backbone and audio backbone	200×6 and 198×40	20 and 20 KB

Table 4.8: Firmware variants for on-device inference. The tensor arena is the fixed memory block that holds the intermediate results of a network.

4.8.2 Model Conversion and Integer Quantization

The trained Keras models store their weights as 32-bit floating-point numbers. For the microcontroller they are converted into the TensorFlow Lite format and quantized to 8-bit integers [40]. In this form, each value is represented by an integer q between -128 and 127, and the original real value r is recovered as

$$r = s \cdot (q - z), \tag{4.3}$$

where the scale s and the zero point z are fixed for each tensor. Quantization reduces the size of the weights by a factor of four. It also allows all computations to be carried out in integer arithmetic, which is the form best supported by TensorFlow Lite Micro on microcontrollers.

Full integer quantization was used, so that not only the weights but also the intermediate results and the input and output of each network are 8-bit integers. To choose suitable scales for the intermediate results, the converter needs to know which value ranges occur in practice. It therefore receives a representative dataset of 200 standardised windows from the training split, which it passes through the network to record the range of every intermediate tensor. The validation data are not used in this step.

The conversion is carried out by an export script. Besides the converted model, it writes all values that the firmware needs to reproduce the preprocessing into C header files, which are listed in Table 4.9. The firmware is then compiled together with these files. Because all constants are produced automatically from the same objects that were used during training, errors caused by copying values by hand are ruled out. Figure 4.5 shows this process.

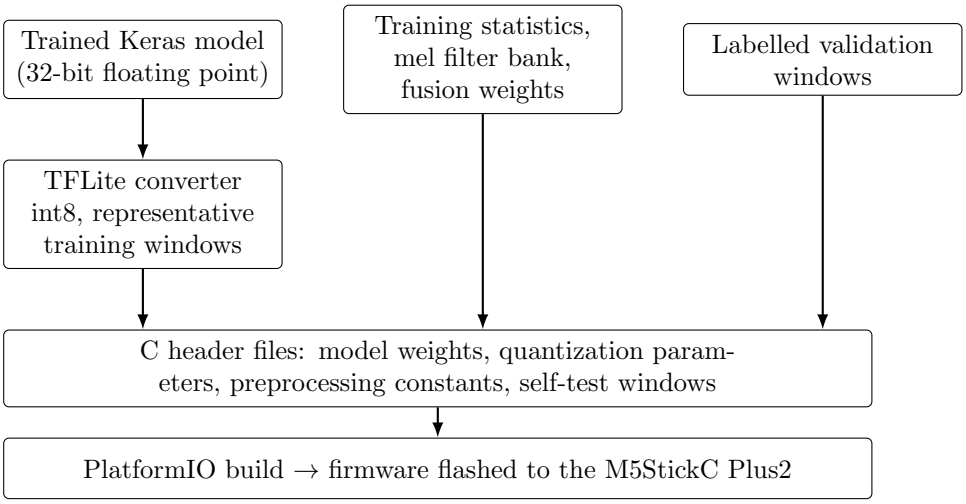


Figure 4.5: Export of a trained model to the device firmware. All constants used on the device are generated automatically from the training pipeline.

Content	**Source**	**Purpose**
Mean and standard deviation of each input channel	training split	standardisation of the live input
Scale and zero point of input and output	TFLite converter	quantization of the input, conversion of the output back to real values
Mel filter bank (257×40)	TensorFlow, as in training	computation of the log-mel spectrogram
Hann window (400 values)	TensorFlow, as in training	weighting of each STFT frame
Class names and fusion weights	training	output and decision-level fusion
One labelled window per class	validation split	self-test at start-up

Table 4.9: Values embedded in the firmware in addition to the model weights.

4.8.3 On-Device Signal Processing

The preprocessing described in Chapter 3 and Section 4.2 was originally implemented in Python and applied offline to complete recordings. On the device, the same steps have to be carried out on a single two-second window, in real time and with little memory.

Inertial data. During data collection, the IMU delivered about 250 samples per second, which were reduced to 80 Hz by averaging consecutive samples, as described in Section 3.6.2. The firmware repeats this step. It collects all IMU samples that arrive during the two-second window and averages them in 160 consecutive groups, which corresponds to the 160 samples of a training window. The result is then resampled to 200 time steps by linear interpolation, standardised with the stored training statistics and quantized with the input scale of the network. To save memory, the last three steps are combined into a single pass that writes the result directly into the input tensor of the network.

Audio data. A two-second audio window consists of 32 000 samples. Stored as 16-bit integers, it would occupy 64 KB, which would not fit into memory together with the networks. The log-mel spectrogram is therefore computed while the audio is being recorded. The firmware reads the microphone in chunks of 160 samples, which corresponds to the 10 ms hop of the STFT, and keeps only the most recent 400 samples, which form the current frame. For each new frame, it multiplies the samples by the Hann window, computes the FFT with a length of 512, applies the mel filter bank and takes the logarithm. Only the resulting 198×40 mel values are kept. The samples are scaled to the range from -1 to 1 beforehand, as in the training pipeline. For the early fusion model, the spectrogram is then resampled to 200 frames and written into the audio channels of the fused input. The decision-level and embedding-level variants pass the 198×40 spectrogram to the audio network unchanged.

Two details of the training pipeline are easy to overlook and were reproduced explicitly. First, the STFT in TensorFlow applies a Hann window to each frame by default, even though the training code does not request it. Second, the mel filter bank of TensorFlow differs slightly from those of other libraries. As listed in Table 4.9, both the window and the filter bank are therefore taken directly from TensorFlow and embedded as tables instead of being recomputed on the device.

4.8.4 Firmware Structure and Inference Loop

After start-up, the firmware initialises the display, the microphone and the networks and runs the self-test described in Section 4.8.5. It then enters a loop, shown in Figure 4.6, that repeats the following steps.

1. A two-second window is recorded. The IMU is read between successive audio chunks, so that both modalities cover the same period of time, as in the synchronised training data.

2. The inertial and acoustic inputs are prepared as described in Section 4.8.3.
3. The network or networks are executed.
4. For decision-level fusion, the output probabilities of both networks are converted back to real values with Equation 4.3 and combined with the per-class weights of Equation 4.2. For embedding-level fusion, the two embeddings are passed to the fusion head.
5. The predicted class and its confidence are shown on the display and sent over the serial port, together with the time required by each step.

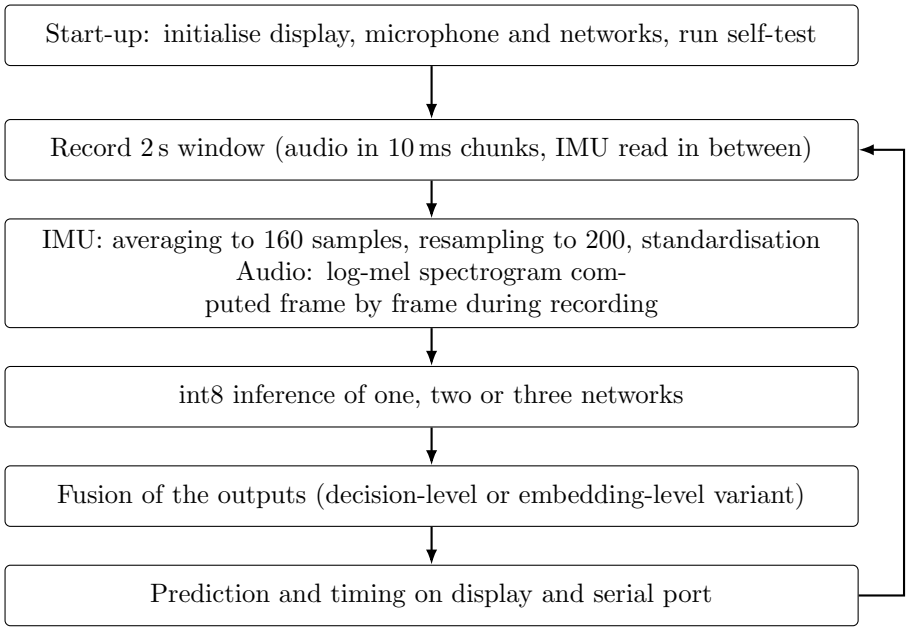


Figure 4.6: Structure of the inference firmware. After the start-up phase, recording, preprocessing, inference and output are repeated continuously.

Recording and inference take place one after the other. While a window is being processed, no new data are recorded, so consecutive windows do not overlap as they do in the training data. A new prediction is therefore available every two seconds plus the processing time.

All memory is reserved statically when the firmware is compiled. TensorFlow Lite Micro places all intermediate tensors of a network in a fixed memory block, the tensor arena, whose size has to be chosen in advance. The sizes in Table 4.8 were chosen so that all buffers fit into the available memory together, and the firmware reports at start-up how much of each arena is actually used. The network weights are not part of the arena, because they remain in flash memory and are read from there during inference. In addition, each network registers only the nine operations it actually uses instead of all available operations, which keeps the program smaller.

4.8.5 Verification of the Deployment Pipeline

When the device produces a wrong prediction, the cause can lie in two places. Either the network itself does not run correctly on the device, or the input it receives differs from the training data. To separate these two cases, each firmware runs a self-test at start-up. It uses one labelled window per class from the validation split. These windows were standardised and quantized in Python with the same constants as on the device and are embedded in the firmware. They are written directly into the input tensor, bypassing the sensors and the on-device preprocessing, and the device reports how many of them are classified correctly. If the self-test succeeds but live predictions are poor, the error must lie in the capture or in the preprocessing.

For the live pipeline, the firmware also prints diagnostic values in every cycle. These include the number of captured IMU samples, the mean acceleration along the three axes and the range of the raw audio samples. The mean acceleration shows how the device is oriented relative to gravity and can be compared with the training data. This makes it possible to detect problems such as a different wearing orientation, whose effect is examined in Chapter 5.

4.9 Evaluation Protocol

4.9.1 Grouped Train/Validation Split

As described in Section 3.7, the windows were created with 50 % overlap, so neighbouring windows share half of their samples. A random split would place such neighbours on both sides of the train/validation boundary. The model would then be evaluated on data that it had partly seen during training, and the measured accuracy would overestimate its performance on new data [15].

To prevent this, the windows are first grouped into activity instances. Consecutive windows with the same label belong to the same group as long as their start times are at most 1.5 s apart, that is, 1.5 times the window stride. Consecutive windows of one uninterrupted activity occurrence start exactly 1 s apart, so the additional 0.5 s serves as a margin. Small irregularities in the window start times therefore do not split one occurrence into several groups, while occurrences separated by a pause or by another activity, which leave a gap of at least one further stride, are still kept apart. The 2565 windows form 125 such groups.

Some of these groups are very long. In recordings with a single long activity, such as walking or carrying a container, one group can contain more than half of all windows of a class. If whole groups were assigned to one side of the split, such a group alone would exceed the intended validation share of its class. Groups with more than 20 windows are therefore cut into blocks of equal size with at most 20 consecutive windows each, which gives 204 blocks in total. The split is then made separately for each class. The blocks of a class are shuffled and considered one after another, and a block is assigned to the validation set only if this brings the number

of validation windows of the class closer to 20 %. The remaining blocks form the training set.

Cutting a group into blocks creates a new boundary inside a continuous activity, where the last window of one block overlaps the first window of the next. The same situation occurs at every transition between two activities. For this reason, every training window whose time span overlaps a validation window is removed from the training set. Since windows start 1 s apart and last 2 s, this affects exactly the windows directly adjacent to a validation block. In total, 56 windows (2.2 %) are discarded in this way. As a result, no training window shares any samples with a validation window, and a fixed random seed makes the split identical for all models, so that their results can be compared directly. Figure 4.7 illustrates the principle.

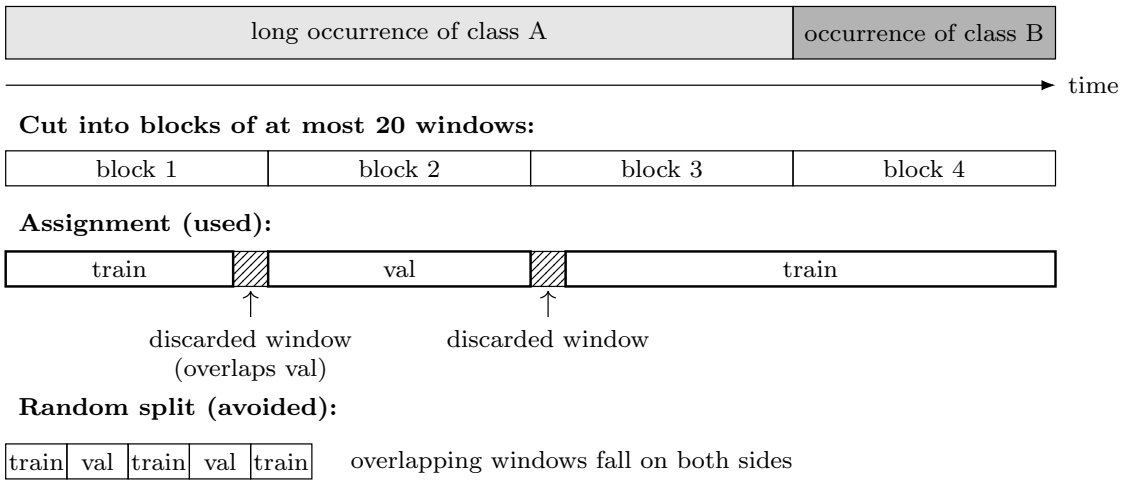


Figure 4.7: Grouped splitting. Long activity occurrences are cut into blocks, and whole blocks are assigned to either the training or the validation set. Training windows that overlap a validation window are discarded. A random split would distribute overlapping windows across both sets.

Table 4.10 shows the resulting split. Because only whole blocks can be moved, the validation share of a class is not exactly 20 %, but it lies between 20 and 22 % for all classes. Overall, 530 windows (20.7 %) form the validation set and 1979 windows (77.2 %) the training set, while 56 windows (2.2 %) are discarded as overlap buffers.

4.9.2 Metrics

Overall accuracy, the share of correctly classified validation windows, is used as the main measure for comparing the models. Because the classes are imbalanced, accuracy alone can hide poor performance on small classes. Precision, recall and F1-score are therefore also reported for each class, together with the macro-averaged F1-score, which gives every class the same weight [8]. Confusion matrices are used to identify which activities are mistaken for one another. Unless stated otherwise,

Class	Training	Validation	Discarded	Total	Val. share
assemble_parts	122	35	9	166	21 %
carry_container	166	45	9	220	20 %
hammering	193	51	6	250	20 %
idle	243	61	3	307	20 %
insert_screws	185	54	7	246	22 %
measuring	244	71	5	320	22 %
place_container	55	15	4	74	20 %
screw_tightening	467	122	7	596	20 %
walk	304	76	6	386	20 %
Total	**1979**	**530**	**56**	**2565**	**21 %**

Table 4.10: Training, validation and discarded windows per class after the grouped split.

all results are reported on the validation set of the grouped split and come from a single training run per model.

4.9.3 On-Device Measurements

The deployed models are evaluated in three ways. First, the accuracy of the quantized models is measured in Python on the full validation set and compared with that of the original floating-point models, which isolates the effect of quantization. Second, the firmware reports the size of each model, the used part of each tensor arena and the time needed for capture, preprocessing and inference, measured with the system timer of the microcontroller. Third, the self-test and live tests with the device worn on the wrist show whether the complete pipeline works under real conditions.

4.10 Chapter Summary

This chapter introduced the models compared in this thesis. Inertial windows are processed as 200×6 time series by a 1D CNN, and audio windows as 198×40 log-mel spectrograms by a 2D CNN, for which three pooling variants were developed to make use of the temporal structure of the sounds. The two modalities are combined at the input level (channel concatenation and unified spectrogram), at the level of learned embeddings, and at the level of class probabilities, where a per-class weighting reflects the different reliability of the two modalities across activities. A random forest on hand-crafted features serves as a classical baseline, and reduced, regularised variants prepare the deployment on the device. All models share the same input normalisation, class weighting, training settings, best-epoch selection and grouped train/validation split. Selected small models are converted to 8-bit

integer arithmetic and executed on the wearable device by firmware that reproduces the complete preprocessing of the training pipeline and checks its own correctness with embedded validation windows. Chapter 5 presents the results of both the offline and the on-device evaluation.

5 Results and Evaluation

6 Discussion

7 Conclusion

Bibliography

[1] Al-Amin, M., Tao, W., Doell, D., Lingard, R., Yin, Z., Leu, M.C., Qin, R.: Action recognition in manufacturing assembly using multimodal sensor fusion. Procedia Manufacturing 39, 158–167 (2019)

[2] Audacity Team: Audacity: Free, open source, cross-platform audio software (2026), `https://www.audacityteam.org/`, accessed 2026. Version [VERIFY]

[3] Baños, O., Galvez, J.M., Damas, M., Pomares, H., Rojas, I.: Window size impact in human activity recognition. Sensors 14(4), 6474–6499 (2014)

[4] Berg, J., Reckordt, T., Richter, C., Reinhart, G.: Action recognition in assembly for human-robot-cooperation using hidden Markov models. Procedia CIRP 76, 205–210 (2018)

[5] Bhattacharya, S., Adaimi, R., Thomaz, E.: Leveraging sound and wrist motion to detect activities of daily living with commodity smartwatches. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 6(2) (2022)

[6] Bonyani, M., Soleymani, M., Wang, C.: Multimodal action recognition for manufacturing assembly task through spatio-temporal knowledge fusion. Information Fusion 132, 104225 (2026)

[7] Bossard Group: SmartTool overview – smart factory assembly (2026), `https://media.bossard.com/-/media/bossard-group/website/documents/smart-factory-assembly/smart-tools/bossard-smarttool-overview-factsheet-en.pdf`, product fact sheet. Accessed 30.10.2026

[8] Bulling, A., Blanke, U., Schiele, B.: A tutorial on human activity recognition using body-worn inertial sensors. ACM Computing Surveys 46(3), 1–33 (2014)

[9] Chatterjee, S., Chakma, A., Gangopadhyay, A., Roy, N., Mitra, B., Chakraborty, S.: LASO: Exploiting locomotive and acoustic signatures over the edge to annotate IMU data for human activity recognition. In: Proceedings of the 2020 International Conference on Multimodal Interaction (ICMI '20) (2020)

[10] Choi, H., Beedu, A., Haresamudram, H., Essa, I.: Multi-stage based feature fusion of multi-modal data for human activity recognition (2022), arXiv:2211.04331

[11] David, R., et al.: TensorFlow Lite Micro: Embedded machine learning for TinyML systems. In: Proceedings of Machine Learning and Systems (MLSys). vol. 3 (2021)

[12] Fraunhofer IIS: Ensure quality in assembly with intelligent tools – pilot project with BMW Regensburg (2017), `https://www.iis.fraunhofer.de/en/pr/2017/201704_LV_Werkzeugtracking.html`, press release, April 2017. Accessed 30.10.2026

[13] Friard, O., Gamba, M.: BORIS: A free, versatile open-source event-logging software for video/audio coding and live observations. Methods in Ecology and Evolution 7(11), 1325–1330 (2016)

[14] Gao, Z., Wang, Y., Chen, J., Xing, J., Patel, S., Liu, X., Shi, Y.: MMTSA: Multi-modal temporal segment attention network for efficient human activity recognition. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 7(3), 96 (2023)

[15] Hammerla, N., Plötz, T.: Let's (not) stick together: Pairwise similarity biases cross-validation in activity recognition. In: Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing (UbiComp '15). pp. 1041–1051 (2015)

[16] Koch, M., Pfitzinger, T., Schlenke, F., Kohlmorgen, F., Groll, R., Wöhrle, H.: Recognition of human activities based on ambient audio and vibration data. IEEE Access 12 (2024)

[17] Koskimäki, H., Huikari, V., Siirtola, P., Laurinen, P., Röning, J.: Activity recognition using a wrist-worn inertial measurement unit: A case study for industrial assembly lines. In: 17th Mediterranean Conference on Control and Automation (MED). pp. 401–405. IEEE (2009)

[18] Kumar, P., Chauhan, S., Awasthi, L.K.: Human activity recognition (HAR) using deep learning: Review, methodologies, progress and future research directions. Archives of Computational Methods in Engineering 31, 179–219 (2024)

[19] Laput, G., Harrison, C.: Sensing fine-grained hand activity with smartwatches. In: Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems (CHI '19). p. Paper 338. ACM (2019)

[20] Lukowicz, P., Ward, J.A., Junker, H., Stäger, M., Tröster, G., Atrash, A., Starner, T.: Recognizing workshop activity using body worn microphones and accelerometers. In: Pervasive Computing (Pervasive 2004). Lecture Notes in Computer Science, vol. 3001. Springer (2004)

[21] M5Stack Technology Co., Ltd.: M5StickC PLUS2 documentation (2026), https://docs.m5stack.com/en/core/M5StickCPLUS2, product documentation. Accessed 2026

[22] Mahfuz, R., Guo, Y., Visser, E., Chinchili, P.: Proactive conversational assistant for a procedural manual task based on audio and IMU (2026), arXiv:2602.15707, preprint

[23] Matin, A., Islam, M.R., Zhu, Y., Wang, X., Huo, H., Xu, G.: Hybrid deep learning for assembly action recognition in smart manufacturing. International Journal of Computer Vision and Signal Processing 14(1), 9–17 (2024)

[24] McFee, B., Raffel, C., Liang, D., Ellis, D.P.W., McVicar, M., Battenberg, E., Nieto, O.: librosa: Audio and music signal analysis in Python. In: Proceedings of the 14th Python in Science Conference. pp. 18–25 (2015)

[25] Mollyn, V., Ahuja, K., Verma, D., Harrison, C., Goel, M.: SAMoSA: Sensing activities with motion and subsampled audio. Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 6(3), 132 (2022)

[26] Nakabayashi, T., Saito, H.: Multimodal human activity recognition on edge devices. In: 2024 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct) (2024)

[27] National Safety Council: MSD solutions lab case study: Wearable sensor pilot at a beverage manufacturing plant (2025), https://www.nsc.org/getmedia/6a48bbb5-7d13-4478-b8e2-496383dbea20/msd-case-study-pepsico.pdf, case study. Accessed 30.10.2026. [VERIFY exact title]

[28] Ouyang, X., Shuai, X., Zhou, J., Shi, I.W., Xie, Z., Xing, G., Huang, J.: Cosmo: Contrastive fusion learning with small data for multimodal human activity recognition. In: Proceedings of the 28th Annual International Conference on Mobile Computing and Networking (MobiCom '22) (2022)

[29] ProGlove (Workaround GmbH): Case study BMW Group: Wearable scanners in dingolfing (2023), https://www.proglove.com/wp-content/uploads/2023/03/case-study-bmw-en.pdf, vendor case study. Accessed 30.10.2026

[30] ProGlove (Workaround GmbH): ProGlove for automotive industries (2026), https://proglove.com/why-proglove/industries/automotive/, vendor web page. Accessed 30.10.2026

[31] Reining, C., Moya Rueda, F., ten Hompel, M., Fink, G., Niemann, F.: Human activity recognition for production and logistics – a systematic literature review. Information 10(8), 245 (2019)

[32] SAP SE: Assembling components with intelligent tools (2026), `https://learning.sap.com/courses/executing-production-processes-in-the-s-factory/assembling-components-with-intelligent-tools`, sAP Digital Manufacturing learning material. Accessed 30.10.2026

[33] Sherafat, B., Rashidi, A., Lee, Y.C., Ahn, C.R.: Automated activity recognition of construction equipment using a data fusion approach (2019), arXiv:1906.02070

[34] Stiefmeier, T., Roggen, D., Ogris, G., Lukowicz, P., Tröster, G.: Wearable activity tracking in car manufacturing. IEEE Pervasive Computing 7(2), 42–50 (2008)

[35] Tao, W., Lai, Z.H., Leu, M.C., Yin, Z.: Worker activity recognition in smart manufacturing using IMU and sEMG signals with convolutional neural networks. Procedia Manufacturing 26, 1159–1166 (2018)

[36] U.S. Government Accountability Office: Workplace safety and health: Ergonomic wearable sensors (technology assessment GAO-25-107213) (2024), `https://gao.gov/assets/gao-25-107213.pdf`, december 2024. Accessed 30.10.2026. [VERIFY exact title]

[37] Ward, J.A., Lukowicz, P., Tröster, G., Starner, T.E.: Activity recognition of assembly tasks using body-worn microphones and accelerometers. IEEE Transactions on Pattern Analysis and Machine Intelligence 28(10), 1553–1567 (2006)

[38] Warden, P.: Speech commands: A dataset for limited-vocabulary speech recognition (2018), arXiv:1804.03209

[39] Yoshimura, N., Morales, J., Maekawa, T., Hara, T.: OpenPack: A large-scale dataset for recognizing packaging works in IoT-enabled logistic environments. In: IEEE International Conference on Pervasive Computing and Communications (PerCom). pp. 90–97 (2024)

[40] Zhou, H., Zhang, X., Feng, Y., Zhang, T., Xiong, L.: Efficient human activity recognition on edge devices using DeepConvLSTM architectures. Scientific Reports 15, 13830 (2025)